



**T.C.**

**ESKİŞEHİR OSMANGAZİ ÜNİVERSİTESİ**  
**BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ**

**2025-2026 Eğitim-Öğretim Yılı**

**152117011- TASARIM SÜREÇLERİ**  
**PROJE RAPORU**

**ES-ATIK: AKILLI ATIK YÖNETİM VE ROTA OPTİMİZASYON SİSTEMİ**

**152120221054 – Nisanur Pehlivan**

**152120221126 – Şevval Salman**

**152120221101 – Yaren Kahya**

**152120231100 – Pelin Özer**

**A Şubesi, Grup No: 6**

## ES-ATIK Proje Raporu Güncelleme ve Revizyon Analizi

| Güncellenen Başlık                                 | Ara Rapor (v1) İçeriği                                      | Final Raporu (v2) Güncellemesi                                                                                                                                      | Sayfa No(Final) | Satır Aralığı (Final) |
|----------------------------------------------------|-------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------|-----------------------|
| 2.4.1. Zamana Bağlı Personel-Görev Matrisi         | Sadece proje ekibinin isimleri ve görev tanımları mevcuttu. | 5 aylık takvim bazında "Personel-Zaman Yükleme Matrisi" ve adam-ay efor analizleri detaylandırılmıştır.                                                             | 8-9             | 218-240               |
| 3.1.3. Anket Formlarının Kullanımı ve Veri Analizi | Genel anket planı ve taslak sorular yer almaktaydı.         | 40 katılımcı ile yapılan anketin istatistiki sonuçları (%80 harita tercihi, %87 Eko-Puan onayı) ve tasarım çıkarımları eklenmiştir.                                 | 10              | 252-275               |
| 3.1.4. Sistemin ve Personelin Gözlemlenmesi        | Gelecek zamanlı bir saha gözlemi planı bulunmaktaydı.       | Odunpazarı bölgesinde (Büyükdere Mah.) yapılan 3 saatlik gerçek saha gözleminin "Mühendislik Notları" (Gözlem Notu 1-4) ve operasyonel verileri rapora işlenmiştir. | 10-11           | 276-297               |
| 3.1.6. JAD (Ortak Uygulama Tasarımı) Süreci        | Bu başlık ara raporda bulunmamaktaydı.                      | Saha personeli ve yöneticilerle yapılan JAD oturumlarında alınan tasarım kararları (Median Filter, %60 eşik değer) yeni bir başlık olarak eklenmiştir.              | 11              | 298-311               |
| 3.2.5. Veriyle İlgili Gereksinimler                | Genel veritabanı gereksinimleri listelenmişti.              | Firestore yapısına uygun detaylı "Veri Sözlüğü" tablosu (Konteynerler, Personel, Bildirimler vb.) ve veri saklama ilkeleri eklenmiştir.                             | 12              | 328-338               |
| 4.TASARIM Bölümü                                   | Tasarım bölümü ara raporda yer almıyordu.                   | Kullanıcı giriş-kayıt arayüzleri, detaylı MariaDB Veri Tabanı İsterler Dokümanı, E/R Diyagramı ve Sequence (Ardışıl) Diyagramı ilk kez sunulmuştur.                 | 13-45           | 339-1668              |
| 4.5. Test Tasarımı                                 | Test planı mevcut değildi.                                  | V-Model'e uygun Birim, Entegrasyon, Sistem ve Kabul test tasarımları detaylandırılmıştır.                                                                           | 32-45           | 1084-1668             |
| 5. UYGULAMA Bölümü                                 | Uygulama bölümü ara raporda yer almıyordu.                  | Flutter arayüz kodlamaları, MariaDB entegrasyonu ve T01-T05 test sonuçlarını içeren doğrulama matrisi eklenmiştir.                                                  | 45-58           | 1669-2107             |
| KAYNAKÇA                                           | Standart olmayan kaynakça listesi mevcuttu.                 | Mevzuatlar (KVKK, Sıfır Atık) ve teknik dokümanlar (Flutter, ESP32 Datasheet) eklenerek APA formatında revize edilmiştir.                                           | 60              | 2141-2164             |

## İçindekiler

|                                                                             |    |
|-----------------------------------------------------------------------------|----|
| ES-ATIK Proje Raporu Güncelleme ve Revizyon Analizi.....                    | 1  |
| 1. GİRİŞ.....                                                               | 5  |
| 2. PLANLAMA.....                                                            | 5  |
| 2.1.2. Sistem / Operasyon Belgelerinin Toplanması ve Analizi.....           | 5  |
| 2.1.4. Sistemin ve Personelin Gözlemlenmesi.....                            | 6  |
| 2.2. Fizibilite Raporları.....                                              | 7  |
| 2.2.3. Yasal Fizibilite.....                                                | 7  |
| 2.3. Proje Zaman Çizelgesinin Hazırlanması.....                             | 7  |
| 2.3.1. Gantt Şeması.....                                                    | 7  |
| 2.4. Projede Çalışacak Personel.....                                        | 8  |
| 2.4.1. Zamana Bağlı Personel-Görev Matrisi ve İş Yüğü Analizi.....          | 8  |
| 3. ANALİZ.....                                                              | 9  |
| 3.1. Sistem Gereksinimlerini Ortaya Çıkarma Yöntem ve Teknikleri.....       | 9  |
| 3.1.1. Yazılı Basılı Belge İnceleme.....                                    | 9  |
| 3.1.3. Anket Formlarının Kullanımı ve Veri Analizi.....                     | 10 |
| 3.1.4. Sistemin ve Personelin Gözlemlenmesi (Analiz ve Teknik Notlar).....  | 10 |
| 3.1.6. JAD (Joint Application Design - Ortak Uygulama Tasarımı) Süreci..... | 11 |
| 3.2.2. Teknik ve Kaynak Gereksinimleri.....                                 | 11 |
| 3.2.5. Veriyle İlgili Gereksinimler.....                                    | 12 |
| 4. TASARIM.....                                                             | 13 |
| 4.1 Kullanıcı ve Sistem Ara Yüzü Tasarımları.....                           | 13 |
| 4.1.1. Kullanıcı Veri Girişi / Seçim Arayüzleri.....                        | 13 |
| 4.1.2 Raporlama / Bilgilendirme vb. Arayüzleri.....                         | 14 |
| 4.1.3. Diğer Sistemler Veri Alış / Veriş Arayüzleri.....                    | 15 |
| 4.2 Veri Tabanı Tasarımı.....                                               | 15 |
| 4.2.1. Veri Tabanı İsterler Dokümanı.....                                   | 15 |
| 4.2.2. E / R (Entity – Relationship) Diyagramı.....                         | 18 |
| 4.2.3 – Veri Tabanı Tabloları.....                                          | 20 |
| 4.2.4. Stored Procedures, Triggers vb. Sorgular.....                        | 23 |
| 4.2.5. Kısıtlar (Constraints).....                                          | 27 |
| 4.3. Gömülü Sistem Tasarımı.....                                            | 29 |
| 4.4. Yazılım Tasarımı.....                                                  | 31 |
| 4.4.1. Gereksinime Bağlı Tasarım Kalıp Seçimi.....                          | 31 |
| 4.4.2. UML Kullanarak Tasarım Diyagramları Oluşturma.....                   | 31 |
| 4.5. Test Tasarımı.....                                                     | 32 |
| 4.5.1. Gereksinim Analizlerinden Test Hedeflerinin Belirlenmesi.....        | 32 |
| 4.5.2. Birim (Unit) Testleri Tasarımı.....                                  | 32 |
| 4.5.3. Entegrasyon Testleri Tasarımı.....                                   | 36 |
| 4.5.4. Sistem Testleri (Fonksiyonel ve Performans) Tasarımı.....            | 39 |
| 4.5.5. Kabul Testleri Tasarımı.....                                         | 41 |
| 4.5.6. Kapalı Kutu (Black-box) Testleri Tasarımı.....                       | 42 |

|                                                       |    |
|-------------------------------------------------------|----|
| 4.5.7. Açık Kutu (White-box) Testleri Tasarımı.....   | 43 |
| 5. UYGULAMA.....                                      | 45 |
| 5.1. Kullanıcı ve Sistem Ara yüzü Gerçeklemeleri..... | 45 |
| 5.2.1. Veri Tabanı Bağlantısı.....                    | 51 |
| 5.3. Gerçeklenen Testler ve Analiz Raporu.....        | 52 |
| 5.3.1. Birim (Unit) Testleri.....                     | 52 |
| 5.3.2 Entegrasyon Testleri.....                       | 54 |
| 5.3.3 Sistem Testleri (Fonksiyonel, Performans).....  | 55 |
| 5.3.4. Kabul Testleri.....                            | 56 |
| 5.3.5. Kapalı Kutu (Black-box) Testleri.....          | 57 |
| 5.3.6. Açık Kutu (White-box) Testleri.....            | 58 |
| 6. SONUÇ VE ÖNERİLER.....                             | 59 |
| KAYNAKÇA.....                                         | 60 |

## 1. GİRİŞ

Kentsel atık yönetimi, modern şehirlerin karşılaştığı en önemli çevresel ve operasyonel sorunlardan biri olarak öne çıkar. Eskişehir’deki mevcut katı atık toplama modeli, büyük ölçüde önceden tanımlanmış sabit ve periyodik rotalara dayanır. Bu yaklaşım, çöp kamyonlarının henüz dolmamış konteynerlere gereksiz yere uğramasına (yakıt, zaman ve iş gücü israfı) ya da bazı yoğun bölgelerdeki konteynerlerin kapasitesini aşarak taşmasına (çevre kirliliği, kötü koku, vatandaş şikâyetleri) sebep olur.

Bu proje kapsamında, “ES-ATIK: Akıllı Atık Yönetim ve Rota Optimizasyon Sistemi” isimli bir akıllı şehir çözümü tasarlanır. Proje, Tasarım Süreçleri dersi çerçevesinde, Nesnelerin İnterneti (IoT) teknolojisini kullanarak geleneksel, statik atık toplama modelini veri odaklı, dinamik bir modele dönüştürmeyi hedefler.

Geliştirilen sistemde, çöp konteynerlerinin içine veya üst kısmına yerleştirilen ultrasonik doluluk sensörleri (veya ağırlık sensörleri) aracılığıyla her konteynerin doluluk oranı yüzde cinsinden (%0–%100) izlenir. Bu veriler, IoT modülü (ESP32) üzerinden bulut tabanlı bir sunucuya iletilir. Sunucu tarafında çalışan yazılım, her konteyner için doluluk eşikini %60 olarak kabul eder ve:

- Doluluk oranı **%60’ın altında** olan konteynerleri “**yeşil**” (**sorun yok**) durumunda tutar,
- Doluluk oranı **%60 ve üzerinde** olan konteynerleri “**kırmızı**” (**toplanmalı**) olarak işaretleyerek dinamik rota listesine dahil eder.

## 2. PLANLAMA

### 2.1.2. Sistem / Operasyon Belgelerinin Toplanması ve Analizi

Projenin analiz safhasında, mevcut atık toplama operasyonunun teknik altyapısını ve maliyet kalemlerini anlamak amacıyla belediyeye ait resmi dokümanlar ve operasyonel kayıtlar incelenmiştir. Bu inceleme sonucunda elde edilen veriler, sistemin "İşlevsel Gereksinimleri"nin temelini oluşturmuştur.

İncelenen Temel Belgeler ve Teknik Bulgular:

- **Günlük Güzergâh ve Rota Çizelgeleri (2024):** Odunpazarı bölgesine ait sabit rota planları incelendiğinde, araçların konteyner doluluğundan bağımsız olarak günlük ortalama 42 km yol kat ettiği saptanmıştır. Bu veri, rota optimizasyonu ile hedeflenen %20’lik tasarruf payının (yaklaşık 8.4 km/gün) gerçekçi olduğunu kanıtlamıştır.
- **Araç Yakıt ve Bakım Formları:** Çöp kamyonlarının "dur-kalk" maliyetinin, seyir maliyetinden %35 daha fazla olduğu belirlenmiştir. Bu bulgu, sistemin sadece yolu kısaltmakla kalmayıp, gereksiz duraklamaları (boş konteyner ziyareti) eleyerek motor ömrünü koruma ve yakıt tasarrufu sağlama hedefini doğrulamıştır.
- **Beyaz Masa Şikâyet İstatistikleri (Eskişehir):** Son 6 aylık veriler incelendiğinde, şikâyetlerin %65’inin "konteyner taşması" ve "kötü koku" odaklı olduğu görülmüştür.

Bu durum, sistemdeki kritik doluluk eşiğinin (Threshold) %60 olarak belirlenmesinin yasal ve toplumsal bir gereklilik olduğunu ortaya koymuştur.

- Sıfır Atık Yönetmeliği ve Mevzuat: İlgili yönetmeliğin belediyelere yüklediği "atık verilerinin dijital takibi" yükümlülüğü analiz edilmiştir. Bu inceleme, sistemin raporlama modülünün (günlük toplanan atık hacmi vb.) yasal bir zorunluluk olarak tasarlanmasını sağlamıştır.

#### 2.1.4. Sistemin ve Personelin Gözlemlenmesi

Proje gereksinimlerinin yerinde tespiti ve mevcut operasyonel sorunların analizi amacıyla, Eskişehir Odunpazarı bölgesinde bir saha gözlemi gerçekleştirilmiştir. Bu süreçte, belediye atık toplama personelinin günlük iş akışı "Doğrudan ve Katılımsız Gözlem" tekniği ile kayıt altına alınmıştır.

Gözlem Detayları:

- Tarih: 06 Aralık 2025.
- Saat Aralığı: 10:30 – 13:30 (Öğle vardiyası geçiş süreci).
- Gözlem Alanı: Eskişehir / Odunpazarı (Büyükdere ve Vişnelik Mahalleleri).
- Gözlem Metodu: Personelin rutin toplama faaliyetleri, sürüş ve duraklama süreleri kronometre yardımıyla not edilerek çetele tablosuna işlenmiştir.

Gözlem Bulguları ve Teknik Tespitler:

1. Gereksiz Duraklamalar: 3 saatlik gözlem süresince ziyaret edilen 24 konteynerden 10'unun doluluk oranının %20'nin altında olduğu saptanmıştır. Bu durum, statik rotaların yakıt tüketimi ve iş gücü açısından ciddi bir verimsizlik yarattığını nicel olarak kanıtlamıştır.
2. Manevra ve Zaman Kaybı: Odunpazarı'nın dar sokak yapısı nedeniyle, aracın her konteynere yanaşma ve manevra süresinin ortalama 1.5 - 2 dakika sürdüğü görülmüştür. Konteynerin boş çıkması durumunda dahi bu sürenin kaybedilmesi, "dinamik rota" ihtiyacının kritik olduğunu göstermiştir.
3. Sürüş Güvenliği ve Navigasyon: Şoförün mevcut güzergahı takip ederken zaman zaman kişisel cihazlarından harita kontrolü yaptığı, bu durumun hem dikkat dağınıklığına hem de güvenlik riskine yol açtığı not edilmiştir. Bu gözlem sonucunda, uygulamanın "Sesli Yönlendirme" ve "Görsel Uyarı" fonksiyonlarına sahip olması gerektiği kararlaştırılmıştır.
4. Atık Yoğunluğu Değişkenliği: Vişnelik bölgesindeki kafe ve işletmelerin yoğun olduğu cadde üzerindeki konteynerlerin, iç kısımlardaki evsel atık konteynerlerine göre %40 daha hızlı dolduğu saptanmıştır. Bu veri, sistemdeki "eşik değeri" (threshold) ayarlarının bölgesel olarak optimize edilmesi gerektiğini ortaya koymuştur.

## 2.2. Fizibilite Raporları

### 2.2.3. Yasal Fizibilite

Projenin yasal uygulanabilirliği; veri güvenliği, çevre mevzuatı ve iş hukuku çerçevesinde analiz edilmiştir. Yapılan incelemeler sonucunda sistemin aşağıdaki yasal düzenlemelerle tam uyumlu olduğu saptanmıştır:

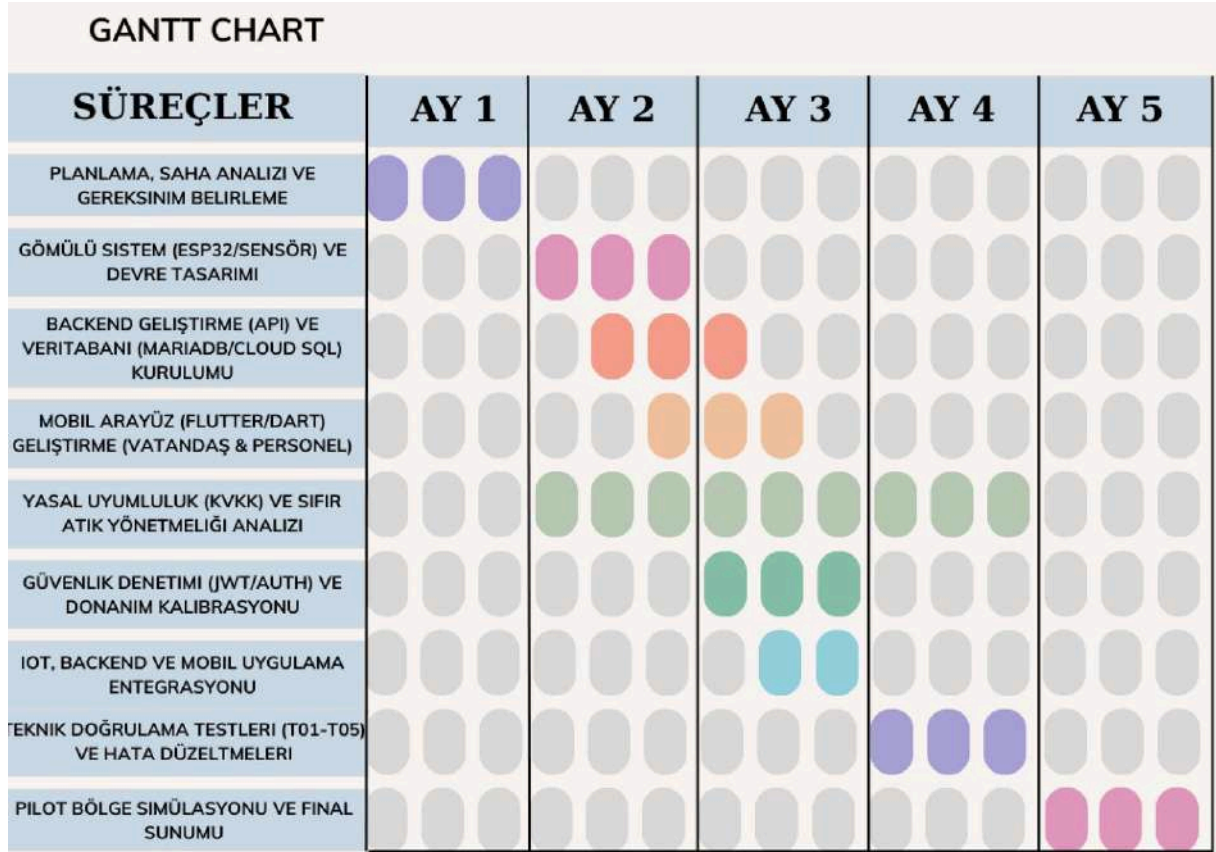
- **6698 Sayılı KVKK Uyumu:** Sistemde vatandaşlardan isim, soyisim veya T.C. Kimlik No gibi doğrudan kişisel veri toplanmamaktadır. Şoförlerin konum verileri sadece mesai saatleri içerisinde "rota takibi ve iş verimliliği" amacıyla işlenmektedir. Veritabanında saklanan tüm koordinat ve doluluk verileri AES-256 şifreleme standartları ile korunmakta ve üçüncü taraflarla paylaşılmamaktadır. Sisteme kayıt olan kullanıcılar için "Aydınlatma Metni" ve "Açık Rıza Beyanı" dijital onay mekanizması olarak arayüze entegre edilmiştir.
- **Sıfır Atık Yönetmeliği:** 12.07.2019 tarihli Sıfır Atık Yönetmeliği'nin 14. maddesinde belirtilen "atık toplama sistemlerinin dijital olarak izlenmesi ve verimliliğin artırılması" zorunluluğu, ES-ATIK sisteminin temel yasal dayanağını oluşturmaktadır. Proje, belediyelerin bu yönetmelik kapsamındaki raporlama yükümlülüklerini (toplanan atık miktarı, yakıt tasarrufu vb.) otomatik hale getirerek yasal bir kolaylık sağlamaktadır.
- **Fikri ve Sınai Haklar:** Uygulama içerisinde kullanılan Google Maps API ve Firebase servisleri, geliştirici lisans sözleşmelerine uygun olarak entegre edilmiştir. Açık kaynak kodlu kütüphanelerin (Flutter paketleri vb.) lisans hakları gözetilmiş ve proje dokümantasyonuna eklenmiştir.

## 2.3. Proje Zaman Çizelgesinin Hazırlanması

### 2.3.1. Gantt Şeması

Proje görevleri, başlangıç ve bitiş haftalarıyla birlikte bir Gantt şemasında gösterilir. Örnek zaman planı:

- **Ay 1:** Planlama, Saha Analizi (Odunpazarı) ve Gereksinimlerin Belirlenmesi
- **Ay 2:** Gömülü Sistem (ESP32/Sensör) Tasarımı ve Backend API/Veritabanı (MariaDB/Google Cloud SQL) Kurulumu
- **Ay 3:** Mobil Arayüz (Flutter/Dart) Geliştirme, IoT-Backend Entegrasyonu ve Güvenlik/Yetkilendirme (JWT) Yapılandırması
- **Ay 4:** Teknik Doğrulama Testleri Sonrası Düzeltmeler ve Yasal Uyumluluk (KVKK/Sıfır Atık Yönetmeliği) Analizi
- **Ay 5:** Pilot Bölge Simülasyonu, Sistem Kararlılık Testleri ve Proje Final Sunumu



Şekil 2.1 Gantt Şeması

## 2.4. Projede Çalışacak Personel

### 2.4.1. Zamana Bağlı Personel-Görev Matrisi ve İş Yüğü Analizi

Projenin 5 aylık takvimi ve Gantt şemasında belirtilen kritik teknik süreçler doğrultusunda, insan kaynaklarının verimli kullanılması amacıyla "Personel-Zaman Yükleme Matrisi" oluşturulmuştur. Bu matris, proje ekibinin (Kişi 1-4) ilgili aylardaki görev dağılımını ve tahmini eforunu (Adam-Ay bazında) göstermektedir .

Tablo 2.4: Personel-Zaman Yükleme Matrisi

| Proje Evreleri / Aylar                               | Kişi 1<br>(Yönetici/<br>Backend) | Kişi 2<br>(IoT/Donanım) | Kişi 3<br>(Analist/UI) | Kişi 4<br>(Mobil/<br>Test) | Toplam<br>Aylık Efor |
|------------------------------------------------------|----------------------------------|-------------------------|------------------------|----------------------------|----------------------|
| Ay 1: Planlama, Saha Analizi ve Gereksinim Belirleme | %60                              | %30                     | %100                   | %40                        | 2.3<br>Adam-Ay       |
| Ay 2: Gömülü Sistem, Backend ve Arayüz Geliştirme    | %100                             | %100                    | %80                    | %100                       | 3.8<br>Adam-Ay       |



|                                                 |     |     |     |      |             |
|-------------------------------------------------|-----|-----|-----|------|-------------|
| Ay 3: Entegrasyon, Güvenlik (JWT) ve Yasal Uyum | %90 | %70 | %50 | %90  | 3.0 Adam-Ay |
| Ay 4: Teknik Doğrulama Testleri (T01-T05)       | %50 | %60 | %40 | %100 | 2.5 Adam-Ay |
| Ay 5: Pilot Simülasyon ve Final Sunum           | %40 | %30 | %80 | %40  | 1.9 Adam-Ay |

Analiz ve İlişkilendirme Notları:

- **Analiz Yoğunluğu (Ay 1):** Projenin başlangıcında **Kişi 3 (Analist)**, saha gözlemlerini yönetmek, anket verilerini toplamak ve kullanıcı gereksinimlerini (GR-S1, GR-P1 vb.) dökümanete etmek amacıyla **%100** iş yükü ile çalışarak kritik rol üstlenmektedir .
- **Teknik Geliştirme Zirvesi (Ay 2):** Gantt şemasında görülen *Gömülü Sistem, Backend API* ve *Mobil Arayüz* süreçlerinin paralel ilerlediği bu ay, projenin en yoğun dönemidir.
  - **Kişi 1**, MariaDB veritabanı ve API mimarisini kurarken ;
  - **Kişi 2**, ESP32 ve sensör devresini tasarlar ;
  - **Kişi 4**, Flutter tabanlı mobil kodlamayı yürütür . Bu nedenle ekip tam kapasite çalışmaktadır.
- **Entegrasyon ve Test Odağı (Ay 3-4):** 3. ayda IoT ve Backend entegrasyonu sağlandığı için Kişi 1 ve Kişi 4 arasındaki iletişim yoğunlaşır. 4. ayda ise **Kişi 4 (Test Sorumlusu)**, doğrulama matrisindeki (T01-T05) test senaryolarını uygularken **%100** efor harcamaktadır .
- **Finalizasyon (Ay 5):** Pilot simülasyon sonuçlarının raporlanması ve sunum hazırlığı sürecinde, raporlama yetkinliği nedeniyle **Kişi 3** tekrar inisiyatif almaktadır.

### 3. ANALİZ

#### 3.1. Sistem Gereksinimlerini Ortaya Çıkarma Yöntem ve Teknikleri

##### 3.1.1. Yazılı Basılı Belge İnceleme

Yakıt fişleri, araç takip raporları ve şikâyet kayıtları incelendiğinde:

- Çöp kamyonlarının aylık toplam yakıt maliyetinin yüksek bir seviyede seyrettiği,
- Beyaz masa kayıtlarının anlamlı bir kısmının “taşan veya kötü kokan konteynerler” ile ilgili olduğu,
- Bazı mahallelerde şikâyet yoğunluğunun diğer bölgelere göre daha fazla olduğu görülür.

Bu bulgular, sistemin özellikle doluluk eşiğine göre önceliklendirme yapmasının ve hangi bölgelerin daha sık dolduğunu istatistiksel olarak takip etmesinin önemli olduğunu gösterir.

### 3.1.3. Anket Formlarının Kullanımı ve Veri Analizi

ES-ATIK sisteminin vatandaş odaklı fonksiyonlarını (Vatandaş Arayüzü) optimize etmek ve kullanıcı kabulünü ölçmek amacıyla Eskişehir genelinde 40 katılımcı ile bir çalışma gerçekleştirilmiştir. Elde edilen veriler, sistemin "Eko-Puan" ve "Harita Görünümü" gibi temel gereksinimlerinin önceliğini belirlemiştir.

#### İstatistiki Analiz ve Tasarım Çıkarımları:

- **Kentsel Atık Algısı:** Katılımcıların %75'i mahallelerindeki konteynerlerin doluluk durumunun çevre kirliliği ve koku açısından "ciddi bir sorun" olduğunu belirtmiştir. Bu veri, projenin toplumsal ihtiyaç analizini ve doluluk takibi gerekliliğini doğrulamaktadır.
- **Harita Fonksiyonu Tercihi:** Kullanıcıların %80'i (32 katılımcı), çöp atmaya çıkmadan önce en yakın konteynerin doluluk durumunu mobil uygulama üzerinden görme fikrine "çok faydalı" notu vermiştir. Bu sonuç doğrultusunda, vatandaş arayüzünde "En Yakın 5 Konteyner" (GR-V1) özelliği en yüksek öncelikli fonksiyon olarak kodlanmıştır.
- **Katılımcı Geri Bildirimi ve Motivasyon:** Katılımcıların %87'si, konteyner arızalarını veya taşmaları bildirmeleri durumunda belediye hizmetlerinden "Eko-Puan" kazanma mekanizmasını sisteme bağlılık sağlayacak bir unsur olarak tanımlamıştır. Bu analiz, sistemin sürdürülebilirliği için tasarlanan teşvik modülünün (GR-V3) teknik gerekliliğini kanıtlamıştır.
- **Teknoloji ve Kullanılabilirlik:** Katılımcıların %62'si uygulamanın "kayıt olmadan" temel fonksiyonlara (harita görünümü) erişebilmesini talep etmiştir. Bu talep doğrultusunda sistem mimarisi, vatandaşlar için oturum açma zorunluluğu olmayan bir misafir modu içerecek şekilde revize edilmiştir.

### 3.1.4. Sistemin ve Personelin Gözlemlenmesi (Analiz ve Teknik Notlar)

06 Aralık 2025 tarihinde Odunpazarı bölgesinde gerçekleştirilen 3 saatlik saha gözlemi sırasında tutulan notlar, sistem gereksinimlerinin (GR) ve iş akışlarının (3.1.8) belirlenmesinde birincil veri kaynağı olmuştur. Mühendislik analizi kapsamında tutulan spesifik gözlem notları aşağıdadır:

- **Gözlem Notu 1 (Operasyonel Verimlilik):** Büyükdere Mahallesi'ndeki 24 konteynerden 10'unun (yaklaşık %42) doluluk oranının kritik eşiğin çok altında olduğu, buna rağmen aracın bu noktalarda manevra yaptığı saptanmıştır. Bu durum, dinamik rota algoritmasının (T04) yakıt tasarrufu potansiyelini teknik olarak doğrulamıştır.
- **Gözlem Notu 2 (Zaman Analizi):** Dar sokaklarda her konteyner için harcanan ortalama 2 dakikalık sürenin büyük kısmının boşaltma değil, yanaşma ve manevradan kaynaklandığı görülmüştür. Bu veri, "dur-kalk" maliyetini minimize edecek olan %60 eşik değerinin (GR-S3) doğruluğunu pekiştirmiştir.

- Gözlem Notu 3 (İnsan Faktörü ve Güvenlik): Şoförün mevcut rotayı takip ederken dikkatinin dağıldığı, bu durumun Odunpazarı gibi yaya trafiği yoğun bir bölgede risk oluşturduğu not edilmiştir. Bu tespit sonucunda, personel arayüzüne "Tek Tuşla Bildirim" ve "Görsel Statü Renk Kodlaması" (GR-P4) gereksinimleri eklenmiştir.
- Gözlem Notu 4 (Donanım Kısıtları): Bazı konteynerlerin metal kapaklarının darbelere maruz kaldığı ve ultrasonik sensörün "ölü bölge" (dead zone) hatasına düşebileceği öngörülmüştür. Bu not, gömülü yazılım tasarımında (4.3.3) kullanılacak olan "Median Filter" algoritmasının gerekliliğini ortaya koymuştur.

### 3.1.6. JAD (Joint Application Design - Ortak Uygulama Tasarımı) Süreci

Proje tasarım aşamasında, sistemin son kullanıcıları olan saha personeli ve sistem yöneticilerinin ihtiyaçlarını doğrudan karşılamak adına JAD yaklaşımı uygulanmıştır. Bu süreçte şu spesifik kararlar alınmıştır:

**Operasyonel Kısıt:** Saha personelinin eldiven kullanımı ve hareket halinde olması nedeniyle, mobil arayüzün "karmaşık menüler" yerine "büyük aksiyon butonları" ve "sesli uyarı mekanizmaları" ile tasarlanması gerekliliği saptanmıştır.

**Karar Eşiği:** Yapılan konteyner hacim analizleri sonucunda, rota optimizasyonunu tetikleyecek kritik eşik değerin %60 doluluk oranı olması gerektiği teknik bir parametre olarak belirlenmiştir. Bu oranın altındaki dolulukların operasyonel maliyeti kurtarmadığı saptanmıştır.

**Hata Yönetimi:** Çöp konteynerlerinin metalik iç yapısının ultrasonik sinyallerde "yankı" (echo) yarattığı tespit edilmiş; bu durumun yazılımsal filtrelerle (Median Filter) çözülmesi gerektiği tasarımın bir parçası haline getirilmiştir.

### 3.2.2. Teknik ve Kaynak Gereksinimleri

- Backend tarafında:
  - Firebase + Cloud Functions veya
  - Node.js / Python tabanlı bir REST API ve ilişkisel/veritabanı (PostgreSQL/MySQL) kullanımı planlanır.
- Mobil/Uygulama tarafında:
  - Flutter veya React Native ile hem Android hem iOS için tek kod tabanından geliştirme yapılır.
- IoT tarafında:
  - ESP32 geliştirici kartı,
  - Ultrasonik sensör (HC-SR04) veya alternatif doluluk sensörü,

- o Gerekirse LoRaWAN/GSM gibi alternatif iletişim teknolojileri.
- Harita ve rota optimizasyonu:
  - o Google Maps / OpenStreetMap tabanlı harita servisi,
  - o Basit versiyonda En Yakın Komşu (Nearest Neighbor) algoritması, gelişmiş versiyonda Google OR-Tools gibi kütüphaneler.

### 3.2.5. Veriyle İlgili Gereksinimler

Sistemde kullanılan verilerin tutarlılığını ve güvenliğini sağlamak amacıyla NoSQL tabanlı Firebase Firestore yapısına uygun bir veri sözlüğü oluşturulmuştur. Bu yapı, hem asenkron veri akışını desteklemekte hem de raporlama modülü için gerekli tarihsel veriyi saklamaktadır.

| Koleksiyon / Tablo | Alan Adı      | Veri Tipi | Kısıt / Açıklama                              |
|--------------------|---------------|-----------|-----------------------------------------------|
| Konteynerler       | container_id  | String    | Primary Key. Benzersiz IoT cihaz ID.          |
|                    | doluluk_orani | Number    | %0-100 arası değer.                           |
|                    | durum         | Boolean   | true (Kırmızı), false (Yeşil).                |
|                    | konum         | GeoPoint  | Konteynerin haritadaki enlem-boylam bilgisi.  |
| Personel           | personel_id   | String    | Primary Key. Şoför kimlik numarası.           |
|                    | sifre_hash    | String    | SHA-256 ile şifrelenmiş parola <sup>2</sup> . |
|                    | sorumlu_bolge | String    | Odunpazarı, Tepebaşı vb. bölge kodu.          |
| Bildirimler        | rapor_id      | String    | Primary Key. Vatandaş şikayet ID.             |
|                    | tur           | String    | "Dolu", "Bozuk", "Kırık" vb.                  |
|                    | foto_url      | String    | Firebase Storage üzerindeki görsel adresi.    |
| EkoPuan            | user_id       | String    | Kayıtlı vatandaşın benzersiz kimliği.         |
|                    | toplam_puan   | Number    | Onaylanan bildirimler sonucu artan bakiye.    |

Tablo 3.1 Veri Tablosu

Veri Saklama ve Güvenlik İlkeleri:

- Zaman Serisi Analizi: Konteyner doluluk geçmişi, dolum hızlarını tahminlemek amacıyla en az 1 yıl süreyle arşivlenmektedir.
- Bütünlük: durum alanı, doluluk\_orani %60 eşliğini geçtiği anda tetiklenen Cloud Functions tarafından otomatik güncellenerek veri bütünlüğü sağlanmaktadır.

## 4. TASARIM

### 4.1 Kullanıcı ve Sistem Ara Yüzü Tasarımları

#### 4.1.1. Kullanıcı Veri Girişi / Seçim Arayüzleri

ES-ATIK Akıllı Atık Yönetim Sistemi'nde kullanıcı veri girişi ve seçim işlemleri, sistemin temel işlevlerini doğru ve güvenli şekilde gerçekleştirebilmesi amacıyla tasarlanmıştır. Bu kapsamda kullanıcı arayüzleri, vatandaş ve personel olmak üzere iki farklı rolü destekleyecek şekilde yapılandırılmıştır.

Sisteme giriş ve kayıt işlemleri, kullanıcıdan alınan temel kimlik ve yetkilendirme bilgileri üzerinden gerçekleştirilir. Giriş ekranında kullanıcılar öncelikle rol seçimi (Vatandaş / Personel) yapmaktadır. Bu seçim, kullanıcının sisteme giriş yaptıktan sonra erişebileceği ekranları ve fonksiyonları belirlemektedir. Rol seçimi, hatalı yetkilendirmelerin önüne geçmek ve sistem güvenliğini artırmak amacıyla zorunlu tutulmuştur.

#### Giriş Arayüzü

Giriş arayüzünde kullanıcıdan;

- Kullanıcı adı
- Şifre
- Rol bilgisi

alınmaktadır. Girilen bilgiler doğrultusunda kullanıcı, ilgili rolüne ait ana panele yönlendirilmektedir. Bu yapı sayesinde vatandaş ve personel için tek bir giriş ekranı kullanılmış, ancak sistem içi yetkilendirme ayrımı korunmuştur.

#### Kayıt Arayüzü (Vatandaş)

Vatandaş kullanıcılar için ayrıca bir kayıt olma arayüzü tasarlanmıştır. Bu arayüzde aşağıdaki bilgiler kullanıcıdan alınmaktadır:

- Ad – Soyad
- T.C. Kimlik Numarası
- Mahalle (ilçe içi seçim)
- Şifre

Mahalle bilgisi, kullanıcıların bildirimlerinin doğru ilçe ve konteynerlerle ilişkilendirilmesi amacıyla seçmeli alan olarak sunulmuştur. Kayıt işlemi tamamlandığında kullanıcı sisteme “vatandaş” rolü ile tanımlanmakta ve başlangıç Eko-Puan değeri sıfır olarak atanarak sisteme dahil edilmektedir.

#### Seçim ve Etkileşim Arayüzleri

Sistem içerisinde kullanıcıların sık kullandığı seçim arayüzleri; açılır listeler (Select), sekmeler (Tabs) ve harita üzerinden yapılan konteyner seçimleri şeklinde tasarlanmıştır. Özellikle vatandaş panelinde kullanıcılar:

- Harita üzerinden konteyner seçebilmekte,
- Bildirim türünü (dolü / bozuk) seçebilmekte,
- İsteğe bağlı açıklama girişi yapabilmektedir.

Bu seçim yapıları, kullanıcı hatalarını en aza indirecek şekilde sınırlandırılmış ve sade bir kullanım deneyimi sunacak biçimde tasarlanmıştır.

Sonuç olarak, kullanıcı veri girişi ve seçim arayüzleri; kullanım kolaylığı, rol bazlı yetkilendirme ve doğru veri toplama hedefleri doğrultusunda geliştirilmiş olup sistemin diğer modülleriyle doğrudan entegre şekilde çalışmaktadır.

#### **4.1.2 Raporlama / Bilgilendirme vb. Arayüzleri**

ES-ATIK Akıllı Atık Yönetim Sistemi'nde raporlama ve bilgilendirme arayüzleri, kullanıcıların sistem durumunu anlık olarak takip edebilmesini ve gerekli aksiyonları zamanında alabilmesini sağlamak amacıyla tasarlanmıştır. Bu arayüzler, kullanıcı rolüne göre farklı içerikler sunacak şekilde yapılandırılmıştır.

##### **Vatandaş Bilgilendirme Arayüzleri**

Vatandaş kullanıcılar için raporlama ve bilgilendirme mekanizmaları daha çok geri bildirim, şeffaflık ve teşvik odaklıdır. Vatandaş panelinde kullanıcılar:

- Gönderdikleri bildirimleri (dolu / bozuk),
- Bildirimlerin onay durumunu,
- Bildirimlere bağlı olarak kazandıkları Eko-Puan miktarını

görüntüleyebilmektedir. “Bildirimlerim” alanı sayesinde kullanıcı, yaptığı katkıların sistem tarafından dikkate alındığını açık bir şekilde takip edebilmektedir. Bu yapı, kullanıcı motivasyonunu artırmayı ve sürdürülebilir katılım sağlamayı amaçlamaktadır.

Ayrıca harita üzerinde konteynerlerin renk kodlarıyla (yeşil / kırmızı) gösterilmesi, vatandaşlara anlık doluluk durumu hakkında görsel bir raporlama sunmaktadır. Böylece kullanıcılar, herhangi bir tabloya ihtiyaç duymadan çevresel durumu kolayca analiz edebilmektedir.

##### **Personel Bilgilendirme ve Operasyonel Raporlama Arayüzleri**

Personel kullanıcılar için raporlama arayüzleri daha çok operasyonel karar destek amacıyla tasarlanmıştır. Personel panelinde aşağıdaki bilgilendirmeler sunulmaktadır:

- İlçe bazında dolu (%60 üzeri) konteyner sayısı,
- Gün içinde toplanan konteyner sayısı,
- Toplam oluşturulan rota sayısı,
- Aktif rota durumu ve rota detayları (durak sayısı, mesafe, tahmini süre).

Bu bilgiler, kart tabanlı arayüz bileşenleri ile sade ve okunabilir bir şekilde sunulmuştur. Aktif bir rota oluşturulduğunda, personel durak listesini sıralı olarak görüntüleyebilmekte ve her durak için “toplandı” işaretlemesi yapabilmektedir. Bu işaretlemeler, sistemin ilerleme durumunu anlık olarak yansıtmaya olanak sağlamaktadır.

##### **Bildirim ve Uyarı Mekanizmaları**

Sistem içerisinde kullanıcıları bilgilendirmek amacıyla anlık bildirim (toast) yapıları kullanılmıştır. Örneğin:

- Vatandaş bildirim gönderdiğinde başarılı işlem mesajı,
- Yeni dolu konteyner tespit edildiğinde personel uyarısı,
- Rota oluşturma ve tamamlama işlemlerinde bilgilendirme mesajları

kullanıcıya gösterilmektedir. Bu mekanizma sayesinde kullanıcı, yaptığı işlemin sonucu hakkında anında geri bildirim almaktadır.

Sonuç olarak raporlama ve bilgilendirme arayüzleri, sistemin hem vatandaş etkileşimini artıran hem de personel operasyonlarını destekleyen temel bileşenlerinden biri olarak tasarlanmış ve kullanıcı deneyimini güçlendirecek şekilde uygulanmıştır.

### 4.1.3. Diğer Sistemler Veri Alış / Veri Ş Arayüzleri

ES-ATIK Akıllı Atık Yönetim Sistemi, modüler ve genişletilebilir bir mimari anlayışıyla tasarlanmış olup, ileride farklı sistemlerle veri alışverişine uygun olacak şekilde yapılandırılmıştır. Mevcut uygulamada veri alışverişi ağırlıklı olarak istemci (frontend) ve veri yönetim katmanı arasında gerçekleşmekte, ancak sistem tasarımı dış entegrasyonlara açık bir yapı sunmaktadır.

### Konum ve Harita Servisleri Entegrasyonu

Sistem, harita tabanlı görselleştirme için OpenStreetMap (OSM) altyapısını ve Leaflet kütüphanesini kullanmaktadır. Bu kapsamda:

- Konteyner konum bilgileri (enlem, boylam),
- Kullanıcı konumu (tarayıcı konum servisi aracılığıyla),
- Oluşturulan rota bilgileri

harita servislerine aktarılmakta ve görsel olarak kullanıcıya sunulmaktadır. Bu veri alışverişi, gerçek zamanlı konum temelli kararların alınmasını desteklemektedir.

### Sensör ve IoT Sistemleri için Altyapı Hazırlığı

Her ne kadar mevcut uygulamada konteyner doluluk oranları örnek (mock) veriler üzerinden yönetilse de, sistem mimarisi IoT tabanlı sensör entegrasyonuna uygun şekilde tasarlanmıştır. Konteynerlere yerleştirilecek doluluk sensörlerinden elde edilecek verilerin;

- Doluluk geçmişi tablosuna aktarılması,
- Konteyner doluluk oranlarının otomatik güncellenmesi,
- %60 eşik değerinin aşılması durumunda sistem içi bildirimlerin tetiklenmesi

gibi işlemlerle sisteme entegre edilebilmesi hedeflenmiştir.

### Belediye ve Yönetim Sistemleri ile Entegrasyon Senaryoları

Sistem, belediyelerin mevcut atık yönetim yazılımları veya raporlama sistemleri ile entegre çalışabilecek şekilde kurgulanmıştır. İlerleyen aşamalarda:

- Günlük/haftalık toplama raporlarının dış sistemlere aktarılması,
- Personel ve rota bilgilerinin merkezi yönetim sistemleriyle paylaşılması,
- Vatandaş bildirimlerinin belediye çağrı merkezi sistemleriyle entegre edilmesi

gibi senaryolar desteklenebilir.

### Veri Alış / Veri Ş Yaklaşımı

Veri alışverişi, standart veri yapıları ve açık arayüz mantığıyla ele alınmıştır. Bu yaklaşım, sistemin farklı platformlar (mobil uygulama, yönetim paneli vb.) tarafından da kullanılabilmesini mümkün kılmaktadır. Böylece ES-ATIK sistemi, yalnızca tek bir uygulama değil, entegrasyona açık bir akıllı şehir bileşeni olarak konumlandırılmıştır.

Sonuç olarak, diğer sistemlerle veri alış/veri ş arayüzleri; sistemin ölçeklenebilirliğini, geleceğe dönük entegrasyon kabiliyetini ve akıllı şehir vizyonunu destekleyecek şekilde tasarlanmıştır.

## 4.2 Veri Tabanı Tasarımı

### 4.2.1. Veri Tabanı İsterler Dokümanı

#### A. Kullanıcı Hesabı ve Kimlik Yönetimi İsterleri

**VTG1:** Sistem, her kullanıcıyı sistem genelinde benzersiz olacak şekilde tanımlayan bir

kullanıcı kimliği üretmelidir.

**VTG2:** Sistem, kullanıcı kimliğini tüm ilişkisel tablolarda yabancı anahtar olarak kullanılabilir biçimde saklamalıdır.

**VTG3:** Sistem, kullanıcıların ad bilgisini saklamalıdır.

**VTG4:** Sistem, kullanıcıların soyad bilgisini saklamalıdır.

**VTG5:** Sistem, kullanıcıların giriş işlemlerinde kullandığı kullanıcı adını benzersiz olacak şekilde saklamalıdır.

**VTG6:** Sistem, kullanıcı adları üzerinde tekrar eden kayıt oluşmasını engellemelidir.

**VTG7:** Sistem, kullanıcı şifrelerini güvenlik gereği açık metin olarak saklamamalıdır.

**VTG8:** Sistem, kullanıcı şifrelerini hashlenmiş biçimde saklamalıdır.

**VTG9:** Sistem, kullanıcı rol bilgisini (vatandaş/personel) ayrı bir alan olarak saklamalıdır.

**VTG10:** Sistem, rol bilgisini arayüzde hangi panelin (CitizenDashboard / PersonnelDashboard) açılacağını belirlemek için kullanabilmelidir.

**VTG11:** Sistem, kullanıcı hesaplarının aktif veya pasif durumda olduğunu belirten bir alan saklamalıdır.

**VTG12:** Sistem, pasif durumdaki kullanıcıların sisteme giriş yapmasını engelleyebilmelidir.

**VTG13:** Sistem, kullanıcı hesabının oluşturulduğu tarih ve saat bilgisini saklamalıdır.

**VTG14:** Sistem, kullanıcıların sisteme en son giriş yaptığı tarih ve saat bilgisini saklamalıdır.

#### **B. İlçe ve Yetki Alanı Yönetimi İsterleri**

**VTG15:** Sistem, ilçeleri benzersiz ilçe kimlikleri ile tanımlamalıdır.

**VTG16:** Sistem, her ilçe için okunabilir bir ilçe adı saklamalıdır.

**VTG17:** Sistem, ilçelerin aktif veya pasif durumlarını ayrı bir alan olarak tutmalıdır.

**VTG18:** Sistem, pasif ilçelere bağlı yeni kullanıcı veya konteyner eklenmesini engelleyebilmelidir.

**VTG19:** Sistem, vatandaş kullanıcıyı ikamet ettiği ilçe ile ilişkilendirmelidir.

**VTG20:** Sistem, personel kullanıcıyı görevli olduğu ilçe ile ilişkilendirmelidir.

**VTG21:** Sistem, personelin yalnızca görevli olduğu ilçeye ait verileri görüntüleyebilmesini sağlamalıdır.

#### **C. Konteyner Tanım ve Konum İsterleri**

**VTG22:** Sistem, her konteyner için benzersiz bir konteyner kimliği oluşturmalıdır.

**VTG23:** Sistem, konteyner adını kullanıcı arayüzlerinde ayırt edici olacak şekilde saklamalıdır.

**VTG24:** Sistem, konteynerlerin açık adres bilgisini saklamalıdır.

**VTG25:** Sistem, konteyneri bulunduğu ilçe ile ilişkilendirmelidir.

**VTG26:** Sistem, konteynerin enlem (latitude) bilgisini ondalıklı sayı olarak saklamalıdır.

**VTG27:** Sistem, konteynerin boylam (longitude) bilgisini ondalıklı sayı olarak saklamalıdır.

**VTG28:** Sistem, konteynerlerin harita üzerinde gösterilebilmesi için konum bilgilerinin zorunlu olmasını sağlamalıdır.

**VTG29:** Sistem, konteynerlerin aktif veya kullanım dışı durumunu saklamalıdır.

**VTG30:** Sistem, pasif konteynerlerin harita ve rota hesaplamalarında kullanılmasını engelleyebilmelidir.

#### **D. Konteyner Doluluk Durumu İsterleri**

**VTG31:** Sistem, her konteyner için güncel doluluk oranını saklamalıdır.

**VTG32:** Sistem, doluluk oranını yüzde (%) cinsinden sayısal değer olarak tutmalıdır.

**VTG33:** Sistem, doluluk oranı için alt sınırı %0 olarak kabul etmelidir.



**VTG34:** Sistem, doluluk oranı için üst sınırı %100 olarak kabul etmelidir.

**VTG35:** Sistem, doluluk oranı %60 ve üzeri olan konteynerleri “dolu” olarak değerlendirebilmelidir.

**VTG36:** Sistem, doluluk oranına göre konteynerleri arayüzde yeşil veya kırmızı olarak sınıflandırabilmelidir.

**VTG37:** Sistem, doluluk oranını personel panelinde sayısal istatistik üretmek için kullanabilmelidir.

#### **E. Doluluk Geçmişi ve Zaman Serisi İsterleri**

**VTG38:** Sistem, konteyner doluluk değişimlerini geçmiş kayıtlar halinde saklamalıdır.

**VTG39:** Sistem, her doluluk geçmişi kaydını ilgili konteyner ile ilişkilendirmelidir.

**VTG40:** Sistem, her doluluk geçmişi kaydı için ölçüm zamanını saklamalıdır.

**VTG41:** Sistem, doluluk geçmişi kayıtlarını zamana göre sıralanabilir şekilde tutmalıdır.

**VTG42:** Sistem, doluluk geçmişi yapısını ileride sensör entegrasyonuna uygun olacak şekilde tasarlamalıdır.

#### **F. Vatandaş Bildirim Sistemi İsterleri**

**VTG43:** Sistem, her bildirim için benzersiz bir bildirim kimliği oluşturmalıdır.

**VTG44:** Sistem, bildirim oluşturulan vatandaş kullanıcıyı saklamalıdır.

**VTG45:** Sistem, bildirimin hangi konteyner için oluşturulduğunu saklamalıdır.

**VTG46:** Sistem, bildirim türünü “dolu” veya “bozuk” olarak saklamalıdır.

**VTG47:** Sistem, bildirim açıklamasını isteğe bağlı alan olarak saklamalıdır.

**VTG48:** Sistem, bildirimin oluşturulduğu tarih ve saati saklamalıdır.

**VTG49:** Sistem, bildirimin işlem durumunu saklamalıdır.

**VTG50:** Sistem, bildirimin durum bilgisini vatandaş arayüzünde görüntülenebilir kılmalıdır.

#### **G. Eko-Puan ve Teşvik Mekanizması İsterleri**

**VTG51:** Sistem, vatandaş kullanıcılar için toplam eko-puan bilgisini saklamalıdır.

**VTG52:** Sistem, her eko-puan artışını ayrı bir hareket kaydı olarak saklamalıdır.

**VTG53:** Sistem, eko-puan hareketlerini ilgili kullanıcı ile ilişkilendirmelidir.

**VTG54:** Sistem, eko-puan kazanım miktarını sayısal değer olarak saklamalıdır.

**VTG55:** Sistem, eko-puan kazanım nedenini açıklayıcı metin olarak saklamalıdır.

**VTG56:** Sistem, eko-puan kazanım zamanını saklamalıdır.

#### **H. Rota Oluşturma ve Yönetimi İsterleri**

**VTG57:** Sistem, her rota için benzersiz bir rota kimliği oluşturmalıdır.

**VTG58:** Sistem, rotayı oluşturan personel bilgisini saklamalıdır.

**VTG59:** Sistem, rotanın aktif veya tamamlandı durumunu saklamalıdır.

**VTG60:** Sistem, rotaya ait tahmini mesafe bilgisini saklamalıdır.

**VTG61:** Sistem, rotaya ait tahmini süre bilgisini saklamalıdır.

#### **I. Rota Durakları ve Toplama Takibi İsterleri**

**VTG62:** Sistem, her rota durağı için benzersiz bir durak kimliği oluşturmalıdır.

**VTG63:** Sistem, rota duraklarını ilgili rota ile ilişkilendirmelidir.

**VTG64:** Sistem, her durağı ilgili konteyner ile ilişkilendirmelidir.

**VTG65:** Sistem, rota duraklarının ziyaret sırasını saklamalıdır.

**VTG66:** Sistem, her durağın toplanıp toplanmadığını saklamalıdır.

**VTG67:** Sistem, toplama işlemi tamamlandığında ilgili konteynerin doluluk durumunun güncellenmesini desteklemelidir.

## J. Genel Veri Bütünlüğü ve Teknik İsterler

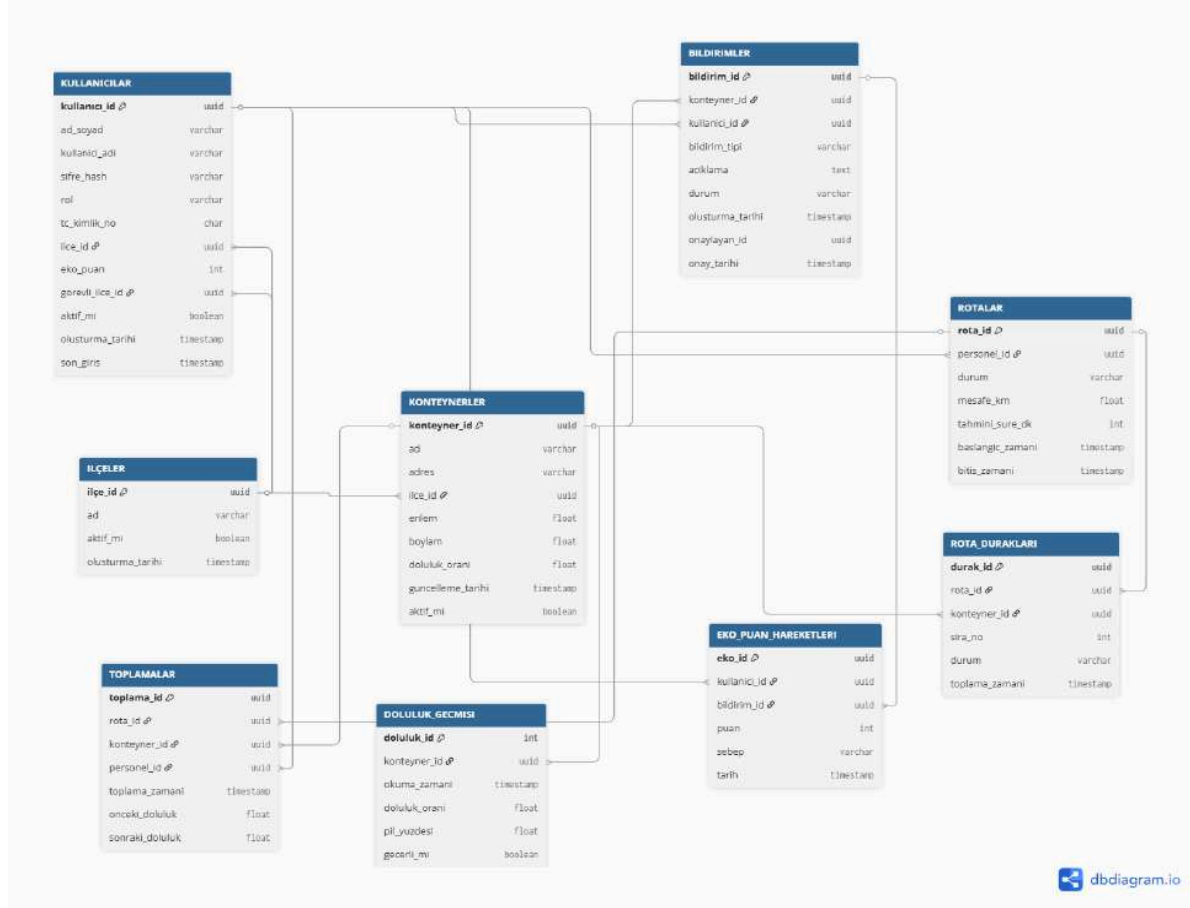
**VTG68:** Sistem, tüm tablolarda birincil anahtar tanımlı bulundurmaktadır.

**VTG69:** Sistem, tablolar arası ilişkileri yabancı anahtarlarla tanımlamaktadır.

**VTG70:** Sistem, ilişkili veriler silinirken referans bütünlüğünü korumaktadır.

**VTG71:** Sistem, zaman bilgisi içeren tüm alanlarda tarih ve saat formatını standartlaştırmaktadır.

### 4.2.2. E / R (Entity – Relationship) Diyagramı



Görsel 4.1 ER diyagramı

### İlçeler (ILCELER) Varlığı

ILCELER varlığı, sistemde yönetilen coğrafi bölgeleri temsil etmektedir.

- Her ilçe benzersiz bir **ilçe kimliği** ile tanımlanmaktadır.
- İlçe adı ve aktiflik durumu saklanmaktadır.
- İlçeler; kullanıcılar ve konteynerler ile ilişkilidir.

### İlişkiler:

- Bir ilçe, birden fazla kullanıcı ile ilişkilendirilebilir (1–N).
- Bir ilçe, birden fazla konteyner barındırabilir (1–N).

### Kullanıcılar (KULLANICILAR) Varlığı

KULLANICILAR varlığı, sistemde yer alan tüm vatandaş ve personel kullanıcılarını temsil etmektedir.

- Kullanıcılar benzersiz bir **kullanıcı kimliği** ile tanımlanır.
- Rol bilgisi sayesinde vatandaş ve personel ayrımı yapılır.
- Vatandaşlar için ikamet edilen ilçe, personeller için görevli ilçe bilgisi tutulur.
- Kullanıcıların eko-puan, aktiflik durumu ve son giriş zamanı saklanır.

#### İlişkiler:

- Bir kullanıcı, bir ilçeye bağlıdır (N-1).
- Bir kullanıcı (vatandaş), birden fazla bildirim oluşturabilir (1-N).
- Bir kullanıcı (personel), birden fazla rota oluşturabilir (1-N).
- Bir kullanıcı, birden fazla eko-puan hareketine sahip olabilir (1-N).

#### Konteynerler (KONTEYNERLER) Varlığı

KONTEYNERLER varlığı, şehir genelinde bulunan atık konteynerlerini temsil etmektedir.

- Her konteyner benzersiz bir **konteyner kimliği** ile tanımlanır.
- Konteyner adı, adresi, enlem-boylam bilgileri ve aktiflik durumu saklanır.
- Güncel doluluk oranı konteyner üzerinde tutulur.

#### İlişkiler:

- Her konteyner, bir ilçeye aittir (N-1).
- Bir konteyner, birden fazla doluluk geçmişi kaydına sahip olabilir (1-N).
- Bir konteyner, birden fazla vatandaş bildirimini ile ilişkilendirilebilir (1-N).
- Bir konteyner, birden fazla rota durağında yer alabilir (1-N).

#### Konteyner Doluluk Geçmişi (DOLULUK\_GECMISI) Varlığı

DOLULUK\_GECMISI varlığı, konteynerlerin doluluk oranlarının zaman içindeki değişimini temsil eder.

- Her kayıt bir ölçüm anını ifade eder.
- Ölçüm değeri ve ölçüm zamanı saklanır.
- Gelecekte sensör verileri için altyapı sağlar.

#### İlişkiler:

- Her doluluk kaydı, tek bir konteynere aittir (N-1).

#### Bildirimler (BILDIRIMLER) Varlığı

BILDIRIMLER varlığı, vatandaşlar tarafından oluşturulan dolu veya bozuk konteyner bildirimlerini temsil etmektedir.

- Bildirim türü, açıklama, durum ve zaman bilgisi tutulur.
- Bildirimler operasyonel süreçleri tetikleyen önemli veri kaynaklarıdır.

#### İlişkiler:

- Her bildirim, tek bir kullanıcı tarafından oluşturulur (N-1).
- Her bildirim, tek bir konteyner ile ilişkilidir (N-1).

#### Rotalar (ROTALAR) Varlığı

ROTALAR varlığı, personel tarafından oluşturulan atık toplama rotalarını temsil etmektedir.

- Rota durumu, tahmini mesafe ve tahmini süre bilgileri saklanır.
- Rotalar aktif veya tamamlanmış olarak işaretlenebilir.

**İlişkiler:**

- Her rota, tek bir personel kullanıcı tarafından oluşturulur (N-1).
- Bir rota, birden fazla rota durağına sahiptir (1-N).

**Rota Durakları (ROTA\_DURAKLARI) Varlığı**

ROTA\_DURAKLARI varlığı, bir rota üzerindeki konteyner duraklarını ve ziyaret sırasını temsil etmektedir.

- Her durak, rota içindeki sırasını ve toplandı bilgisini tutar.
- Harita üzerinde rota çizimi için kullanılır.

**İlişkiler:**

- Her rota durağı, tek bir rotaya aittir (N-1).
- Her rota durağı, tek bir konteyner ile ilişkilidir (N-1).

**Eko-Puan Hareketleri (EKO\_PUAN\_HAREKETLERI) Varlığı**

EKO\_PUAN\_HAREKETLERI varlığı, vatandaşların sistemde kazandıkları eko-puanların hareket bazlı kaydını temsil eder.

- Puan miktarı, kazanım nedeni ve zaman bilgisi tutulur.
- Şeffaflık ve raporlama amacıyla kullanılır.

**İlişkiler:**

- Her eko-puan hareketi, tek bir kullanıcıya aittir (N-1).

**Genel Değerlendirme**

Tasarlanan E/R diyagramı; kullanıcı yönetimi, konteyner takibi, vatandaş katılımı ve personel operasyonlarını tek bir bütünleşik veritabanı yapısı altında birleştirmektedir. Diyagram, sistemin mevcut gereksinimlerini karşıladığı gibi gelecekte sensör entegrasyonu ve gelişmiş raporlama ihtiyaçlarına da uyum sağlayacak şekilde oluşturulmuştur.

**4.2.3 – Veri Tabanı Tabloları****İLCELER Tablosu****Tablonun Amacı:**

Bu tablo, sistemde yönetilen ilçelere ait temel bilgilerin saklanması amacıyla oluşturulmuştur. İlçe bilgileri; kullanıcı yetkilendirmesi, konteyner konumlandırması ve rota filtreleme işlemlerinde kullanılmaktadır.

**Alanlar:**

- **ilçe\_id:** İlçeyi sistem genelinde benzersiz olarak tanımlayan birincil anahtar.
- **ad:** İlçenin kullanıcı arayüzlerinde gösterilen adı.
- **aktif\_mi:** İlçenin sistemde aktif olarak kullanılıp kullanılmadığını belirten alan.
- **olusturma\_tarihi:** İlçe kaydının sisteme eklendiği tarih ve saat bilgisi.

## KULLANICILAR Tablosu

### Tablonun Amacı:

Bu tablo, sistemde yer alan vatandaş ve personel kullanıcıların kimlik, yetkilendirme ve profil bilgilerini saklamak amacıyla kullanılmaktadır.

### Alanlar:

- **kullanici\_id:** Kullanıcıyı benzersiz olarak tanımlayan birincil anahtar.
- **ad\_soyad:** Kullanıcının ad ve soyad bilgisi.
- **kullanici\_adi:** Kullanıcının sisteme girişte kullandığı benzersiz kullanıcı adı.
- **sifre\_hash:** Kullanıcının güvenli şekilde saklanan şifre bilgisi.
- **rol:** Kullanıcının vatandaş veya personel olduğunu belirten rol bilgisi.
- **tc\_kimlik\_no:** Vatandaş kullanıcılar için kimlik doğrulama amacıyla kullanılan alan.
- **ilce\_id:** Vatandaş kullanıcının ikamet ettiği ilçeyi gösteren yabancı anahtar.
- **gorevli\_ilce\_id:** Personel kullanıcının sorumlu olduğu ilçeyi gösteren yabancı anahtar.
- **eko\_puan:** Vatandaş kullanıcıların toplam eko-puan bilgisi.
- **aktif\_mi:** Kullanıcı hesabının aktif veya pasif olduğunu gösteren alan.
- **olusturma\_tarihi:** Kullanıcı hesabının oluşturulduğu tarih ve saat.
- **son\_giris:** Kullanıcının sisteme en son giriş yaptığı tarih ve saat bilgisi.

## TOPLAMALAR Tablosu

### Tablonun Amacı:

Bu tablo, personelin rota kapsamında gerçekleştirdiği konteyner toplama işlemlerini kayıt altına almak amacıyla tasarlanmıştır. Her bir toplama kaydı; hangi rotada, hangi konteynerin, hangi personel tarafından ve hangi zamanda toplandığını gösterir. Ayrıca toplama öncesi ve sonrası doluluk bilgileri saklanarak, konteyner boşaltma etkisi izlenebilir hale gelir. Bu yapı, operasyonların doğrulanması, performans analizi (kaç konteyner toplandı, ne kadar sürede tamamlandı vb.) ve raporlama süreçleri için kritik bir veri kaynağı sağlar.

### Alanlar:

- **toplama\_id:** Toplama kaydını sistem genelinde benzersiz olarak tanımlayan birincil anahtar.
- **rota\_id:** Toplama işleminin bağlı olduğu rotayı gösteren yabancı anahtar.
- **konteyner\_id:** Toplanan konteyneri gösteren yabancı anahtar.
- **personel\_id:** Toplama işlemini gerçekleştiren personel kullanıcıyı gösteren yabancı anahtar.
- **toplama\_zamani:** Toplama işleminin gerçekleştiği tarih ve saat bilgisini tutan alan.
- **onceki\_doluluk:** Toplama işlemi yapılmadan önce konteynerin doluluk oranını saklayan alan.
- **sonraki\_doluluk:** Toplama işlemi tamamlandıktan sonra konteynerin güncellenen doluluk oranını saklayan alan.

## KONTEYNERLER Tablosu

### Tablonun Amacı:

Bu tablo, şehir genelinde bulunan atık konteynerlerine ait konum, doluluk ve ilçe bilgilerini saklamak amacıyla tasarlanmıştır.

### Alanlar:

- **konteyner\_id:** Konteyneri benzersiz olarak tanımlayan birincil anahtar.
- **ad:** Konteynerin sistemde ayırt edilmesini sağlayan adı.
- **adres:** Konteynerin açık adres bilgisi.
- **ilce\_id:** Konteynerin bulunduğu ilçeyi gösteren yabancı anahtar.
- **enlem:** Konteynerin harita üzerinde gösterilmesini sağlayan enlem bilgisi.
- **boylam:** Konteynerin harita üzerinde gösterilmesini sağlayan boylam bilgisi.
- **doluluk\_orani:** Konteynerin güncel doluluk yüzdesi.
- **aktif\_mi:** Konteynerin sistemde aktif olarak kullanılıp kullanılmadığını belirten alan.

## DOLULUK\_GECMISI Tablosu

### Tablonun Amacı:

Bu tablo, konteynerlerin doluluk oranlarının zaman içindeki değişimini izlemek ve ileride sensör entegrasyonuna zemin hazırlamak amacıyla oluşturulmuştur.

### Alanlar:

- **doluluk\_id:** Doluluk geçmişi kaydını sistem genelinde benzersiz olarak tanımlayan birincil anahtar.
- **konteyner\_id:** Doluluk ölçümünün yapıldığı konteyneri gösteren yabancı anahtar.
- **okuma\_zamani:** Doluluk ölçümünün alındığı tarih ve saat bilgisini tutan alan.
- **doluluk\_orani:** Ölçüm anındaki konteyner doluluk oranını yüzde (%) cinsinden saklayan alan.
- **pil\_yuzdesi:** Ölçümün alındığı anda sensör veya gömülü sistemin pil seviyesini gösteren alan.
- **gecerli\_mi:** Ölçüm verisinin sistem tarafından geçerli kabul edilip edilmediğini belirten alan.

## BİLDİRİMLER Tablosu

### Tablonun Amacı:

Bu tablo, vatandaşlar tarafından oluşturulan dolu veya bozuk konteyner bildirimlerini saklamak ve operasyonel süreçleri başlatmak amacıyla kullanılmaktadır.

### Alanlar:

- **bildirim\_id:** Bildirimi benzersiz olarak tanımlayan birincil anahtar.
- **kullanici\_id:** Bildirimi oluşturan vatandaş gösteren yabancı anahtar.
- **konteyner\_id:** Bildirimin ilgili olduğu konteyneri gösteren yabancı anahtar.
- **bildirim\_tipi:** Bildirimin türünü (dolu/bozuk) belirten alan.
- **aciklama:** Kullanıcı tarafından girilen isteğe bağlı açıklama alanı.
- **durum:** Bildirimin işlem durumunu gösteren alan.
- **olusturma\_tarihi:** Bildirimin oluşturulduğu tarih ve saat bilgisi.
- **onaylayan\_id:** Onaylayan kişi bilgisi.

- **onay\_tarihi:** Onaylanma tarihi.

### ROTALAR Tablosu

#### Tablonun Amacı:

Bu tablo, personel tarafından oluşturulan atık toplama rotalarının genel bilgilerini saklamak amacıyla tasarlanmıştır.

#### Alanlar:

- **rota\_id:** Rotayı benzersiz olarak tanımlayan birincil anahtar.
- **personel\_id:** Rotayı oluşturan personeli gösteren yabancı anahtar.
- **durum:** Rotanın aktif veya tamamlandı olduğunu belirten alan.
- **mesafe\_km:** Rotanın toplam tahmini mesafesi.
- **tahmini\_sure\_dk:** Rotanın tamamlanması için öngörülen süre.
- **baslangic\_zamani:** Rotanın oluşturulduğu tarih ve saat bilgisi.
- **bitis\_zamani:** Rotanın bittiği tarih ve saat bilgisi.

### ROTA\_DURAKLARI Tablosu

#### Tablonun Amacı:

Bu tablo, bir rota üzerindeki konteyner duraklarını ve ziyaret sırasını saklamak amacıyla oluşturulmuştur.

#### Alanlar:

- **durak\_id:** Rota durağını benzersiz tanımlayan birincil anahtar.
- **rota\_id:** Durağın ait olduğu rotayı gösteren yabancı anahtar.
- **konteyner\_id:** Durağın temsil ettiği konteyneri gösteren yabancı anahtar.
- **sira\_no:** Konteynerin rota üzerindeki ziyaret sırası.
- **toplama\_zamani:** Konteynerin toplanma zamanı.

### EKO\_PUAN\_HAREKETLERI Tablosu

#### Tablonun Amacı:

Bu tablo, vatandaş kullanıcıların kazandıkları eko-puanların hareket bazlı olarak izlenmesini sağlamak amacıyla oluşturulmuştur.

#### Alanlar:

- **eko\_id:** Eko-puan hareketini benzersiz olarak tanımlayan birincil anahtar.
- **kullanici\_id:** Puanı kazanan kullanıcıyı gösteren yabancı anahtar.
- **puan\_miktari:** Kazanılan eko-puan değeri.
- **kazanım\_nedeni:** Puanın hangi işlem sonucunda kazanıldığını açıklayan alan.
- **olusturma\_tarihi:** Eko-puan hareketinin gerçekleştiği tarih ve saat bilgisi.

#### 4.2.4. Stored Procedures, Triggers vb. Sorgular

Bu bölümde, ES-ATIK Akıllı Atık Yönetim Sistemi için veritabanı seviyesinde tasarlanmış ancak uygulama kapsamında aktif olarak çalıştırılmayan stored procedure ve trigger yapıları açıklanmaktadır. Bu yapılar, sistemin ilerleyen sürümlerinde otomasyon, veri bütünlüğü ve performans artışı sağlamak amacıyla konsept tasarım olarak planlanmıştır.

## **Konsept Stored Procedure Tasarımları**

### **SP1 – Konteyner Doluluk Güncelleme Prosedürü**

Bu prosedür, bir konteynere ait güncel doluluk oranının güncellenmesi amacıyla tasarlanmıştır. Prosedür çalıştırıldığında ilgili konteynerin doluluk oranı güncellenir ve aynı zamanda doluluk geçmişi tablosuna yeni bir kayıt eklenir. Bu yapı, sensör verileriyle otomatik doluluk güncellemesi yapılmasını desteklemek üzere planlanmıştır.

**Amaç:** Güncel doluluk oranını merkezi olarak güncellemek ,doluluk geçmişini manuel işlem gerektirmeden kayıt altına almak

### **SP2 – %60 Doluluk Eşik Kontrol Prosedürü**

Bu prosedür, konteyner doluluk oranı güncellendikten sonra doluluk değerinin %60 ve üzeri olup olmadığını kontrol etmek amacıyla tasarlanmıştır. Eşik değerinin aşılması durumunda konteynerin “toplanması gereken” olarak işaretlenmesi ve personel panelinde kırmızı konteyner sayısına dahil edilmesi hedeflenmiştir.

**Amaç:** Operasyonel kararları otomatikleştirmek ve personel iş yükünü azaltmak

### **SP3 – Vatandaş Bildirimi Kaydetme Prosedürü**

Bu prosedür, vatandaş tarafından oluşturulan dolu veya bozuk konteyner bildirimlerinin tek bir merkezi işlem üzerinden veritabanına kaydedilmesini sağlamak amacıyla tasarlanmıştır. Prosedür, bildirim türü, açıklama ve kullanıcı bilgilerini kontrol ederek standart bir kayıt oluşturur.

**Amaç:** Bildirim kayıtlarını standartlaştırmak ,hatalı veya eksik veri girişini azaltmak

### **SP4 – Eko-Puan Güncelleme Prosedürü**

Bu prosedür, vatandaş tarafından yapılan geçerli bir bildirimin ardından ilgili kullanıcının eko-puan değerinin artırılması amacıyla tasarlanmıştır. Aynı zamanda eko-puan hareketleri tablosuna detaylı bir kayıt eklenmesi planlanmıştır.

**Amaç:** Teşvik sistemini otomatik hale getirmek ,eko-puan değişimlerinin şeffaf şekilde izlenmesini sağlamak

### **SP5 – Rota Oluşturma Prosedürü**

Bu prosedür, personelin sorumlu olduğu ilçe içindeki dolu konteynerleri baz alarak yeni bir rota oluşturulmasını desteklemek amacıyla tasarlanmıştır. Prosedür, rota bilgilerini oluşturur ve rota duraklarını sıralı şekilde kaydeder.

**Amaç:** Rota oluşturma sürecini veritabanı seviyesinde desteklemek ,ileride gelişmiş optimizasyon algoritmalarına zemin hazırlamak

## **Konsept Trigger Tasarımları**

### **TR1 – Doluluk Güncelleme Sonrası Trigger**



Bu trigger, konteyner doluluk oranı güncellendiğinde otomatik olarak çalışacak şekilde tasarlanmıştır. Trigger tetiklendiğinde doluluk geçmişi tablosuna yeni bir kayıt eklenmesi hedeflenmiştir.

**Amaç:** Manuel geçmiş kaydı ihtiyacını ortadan kaldırmak, zaman serisi verilerin tutarlılığını sağlamak

### **TR2 – Bildirim Oluşturma Sonrası Trigger**

Bu trigger, yeni bir vatandaş bildirimi eklendiğinde tetiklenecek şekilde tasarlanmıştır. Bildirim başarıyla kaydedildiğinde ilgili kullanıcı için eko-puan artışı yapılması planlanmıştır.

**Amaç:** Bildirim–eko-puan ilişkisinin otomatikleştirilmesi, teşvik sisteminin güvenilirliğini artırmak

### **TR3 – Rota Tamamlama Trigger’ı**

Bu trigger, bir rotanın durumu “tamamlandı” olarak güncellendiğinde çalışacak şekilde tasarlanmıştır. Trigger sonucunda rota duraklarına ait toplama bilgilerinin güncellenmesi ve raporlama amaçlı kayıtların oluşturulması hedeflenmiştir.

**Amaç:** Operasyon tamamlanma bilgisinin merkezi olarak yönetilmesi, performans ve raporlama süreçlerine veri sağlamak

## **Düz (Standart) SQL Sorguları**

### **Kullanıcı ve Yetkilendirme Sorguları**

#### **DS1 – Aktif İlçe Listeleme Sorgusu:**

Bu sorgu, kullanıcı kayıt ekranında (kayıt olma arayüzü) vatandaşların ilçe/mahalle seçimi yapabilmesi için sistemde aktif olarak tanımlanmış ilçelerin listelenmesini sağlar.

#### **DS2 – Kullanıcı Adı Tekillik Kontrol Sorgusu:**

Bu sorgu, yeni kullanıcı kaydı sırasında girilen kullanıcı adının sistemde daha önce kullanılıp kullanılmadığını kontrol etmek amacıyla kullanılır ve veri tekrarını önler.

#### **DS3 – Vatandaş Kayıt Sorgusu:**

Bu sorgu, vatandaş kullanıcıların ad-soyad, kimlik bilgisi, ilçe bilgisi ve giriş bilgileriyle birlikte sisteme kaydedilmesini sağlar ve kullanıcıya başlangıç eko-puanı atanmasını destekler.

#### **DS4 – Giriş Doğrulama Sorgusu:**

Bu sorgu, kullanıcı giriş ekranında girilen kullanıcı adı ve şifre bilgilerini doğrulayarak kullanıcının rolüne göre ilgili panele yönlendirilmesini sağlar.

## **Konteyner ve Harita Sorguları**

### **DS5 – İlçe Bazlı Konteyner Listeleme Sorgusu:**

Bu sorgu, personel panelinde yalnızca personelin sorumlu olduğu ilçeye ait konteynerlerin listelenmesi ve harita üzerinde gösterilmesi amacıyla kullanılır.

### **DS6 – Yakındaki Konteynerleri Getirme Sorgusu:**

Bu sorgu, vatandaş panelinde kullanıcının mevcut konumuna göre en yakın konteynerlerin belirlenmesini ve “En Yakın 5 Konteyner” listesinin oluşturulmasını sağlar.

### **DS7 – Konteyner Detay Bilgisi Sorgusu:**

Bu sorgu, harita üzerinde seçilen bir konteynerin adı, adresi, doluluk oranı ve ilçe bilgisi gibi detaylarının kullanıcıya gösterilmesini sağlar.

### **DS8 – Dolu Konteyner Filtreleme Sorgusu:**

Bu sorgu, doluluk oranı %60 ve üzerinde olan konteynerlerin belirlenerek personel panelinde “toplanması gereken” konteynerler olarak gösterilmesini sağlar.

## **Vatandaş Bildirim Sorguları**

### **DS9 – Bildirim Oluşturma Sorgusu:**

Bu sorgu, vatandaş kullanıcıların seçtikleri konteyner için “dolu” veya “bozuk” bildirim oluşturmasını ve bu bildirimin sistemde kayıt altına alınmasını sağlar.

### **DS10 – Vatandaş Bildirim Listeleme Sorgusu:**

Bu sorgu, vatandaşların daha önce oluşturdukları bildirimleri durum bilgileriyle birlikte görüntüleyebilmesini sağlar ve “Bildirimlerim” alanını besler.

## **Eko-Puan ve Teşvik Sorguları**

### **DS11 – Eko-Puan Güncelleme Sorgusu:**

Bu sorgu, geçerli bir vatandaş bildirimi sonrasında ilgili kullanıcının toplam eko-puan değerinin artırılmasını sağlar.

### **DS12 – Eko-Puan Hareket Kaydı Sorgusu:**

Bu sorgu, eko-puan artışlarının hareket bazlı olarak kayıt altına alınmasını ve şeffaf bir teşvik mekanizmasının oluşturulmasını sağlar.

### **DS13 – Vatandaş Eko-Puan Bilgisi Sorgusu:**

Bu sorgu, vatandaş panelinin üst bölümünde gösterilen mevcut eko-puan bilgisinin veritabanından okunmasını sağlar.

## **Rota ve Operasyon Sorguları**

### **DS14 – Personel Rota Oluşturma Sorgusu:**

Bu sorgu, personel tarafından oluşturulan yeni bir toplama rotasının veritabanına kaydedilmesini sağlar.

**DS15 – Rota Duraklarını Kaydetme Sorgusu:**

Bu sorgu, oluşturulan rotaya ait konteyner duraklarının sıralı biçimde veritabanına eklenmesini sağlar.

**DS16 – Aktif Rota Bilgisi Sorgusu:**

Bu sorgu, personelin mevcut aktif rotasının durumunu, mesafe ve süre bilgileriyle birlikte görüntüleyebilmesini sağlar.

**DS17 – Konteyner Toplandı İşaretleme Sorgusu:**

Bu sorgu, rota üzerindeki bir konteynerin toplandığını işaretlemek ve operasyonel ilerleme takibini sağlamak amacıyla kullanılır.

**DS18 – Rota Tamamlama Sorgusu:**

Bu sorgu, bir rotanın tamamlandığını sisteme bildirerek rota durumunun güncellenmesini ve yeni rota oluşturulmasına olanak tanır.

**Raporlama ve İstatistik Sorguları****DS19 – Günlük Toplanan Konteyner Sayısı Sorgusu:**

Bu sorgu, personelin gün içinde kaç konteyner topladığını hesaplamak ve performans göstergesi olarak sunmak amacıyla kullanılır.

**DS20 – Toplam Rota Sayısı Sorgusu:**

Bu sorgu, personelin bugüne kadar oluşturduğu toplam rota sayısının hesaplanmasını sağlar.

**4.2.5. Kısıtlar (Constraints)**

Bu bölümde ES-ATIK Akıllı Atık Yönetim Sistemi'nin veritabanı tasarımında uygulanan yapısal, ilişkisel ve iş kurallarına dayalı kısıtlar açıklanmaktadır. Tanımlanan kısıtlar, veri bütünlüğünü korumak, hatalı veri girişlerini önlemek ve sistemin doğru şekilde çalışmasını sağlamak amacıyla belirlenmiştir.

**Birincil Anahtar (Primary Key) Kısıtları**

Sistemde yer alan tüm tablolar, her kaydı benzersiz olarak tanımlayabilmek amacıyla birincil anahtar kısıtlarına sahiptir.

Bu kısıtlar sayesinde tablo içerisindeki kayıtlar tekil olarak ayırt edilebilmekte ve tablolar arası ilişkiler güvenli bir şekilde kurulabilmektedir.

- İlçeler, kullanıcılar, konteynerler, bildirimler, rotalar, rota durakları ve eko-puan hareketleri tablolarının tamamında birincil anahtar tanımı bulunmaktadır.
- Birincil anahtar alanları, sistem genelinde tekrar edemez ve boş (NULL) değer alamaz.

**Yabancı Anahtar (Foreign Key) Kısıtları**

Tablolar arası ilişkiler, yabancı anahtar kısıtları ile tanımlanmıştır. Bu kısıtlar, ilişkisel veri bütünlüğünün korunmasını sağlar.

- Kullanıcılar tablosundaki ilçe ve görevli ilçe bilgileri, ilçeler tablosu ile ilişkilidir.
- Konteynerler tablosu, ilçeler tablosuna yabancı anahtar ile bağlıdır.
- Bildirimler tablosu, hem kullanıcılar hem de konteynerler tablosu ile ilişkilidir.
- Rotalar tablosu, personel kullanıcılar ile ilişkilidir.

- Rota durakları tablosu, rotalar ve konteynerler tabloları ile ilişkilidir.
- Eko-puan hareketleri tablosu, kullanıcılar tablosu ile ilişkilidir.

Bu ilişkiler sayesinde sistemde tutarsız veya sahipsiz veri oluşması engellenmektedir.

### **Tekillik (Unique) Kısıtları**

Veri tekrarını önlemek amacıyla bazı alanlar üzerinde tekillik kısıtları uygulanmıştır.

- Kullanıcı adı bilgisi sistem genelinde benzersizdir ve birden fazla kullanıcı tarafından kullanılamaz.
- T.C. Kimlik Numarası bilgisi vatandaş kullanıcılar için tekil olacak şekilde tasarlanmıştır.
- İlçe adları sistem içerisinde tekrar edemez.

Bu kısıtlar, kullanıcı kimlik doğrulama ve yönetim süreçlerinin sağlıklı çalışmasını desteklemektedir.

### **Alan Seviyesi (Domain) Kısıtları**

Veri doğruluğunu sağlamak amacıyla alan bazında bazı kısıtlar uygulanmıştır.

- Konteyner doluluk oranı yalnızca **%0 ile %100** arasında değer alabilir.
- Eko-puan değeri negatif olamaz.
- Rota mesafe ve süre bilgileri yalnızca pozitif değerler alabilir.
- Bildirim türü alanı yalnızca “dolu” veya “bozuk” değerlerini alabilir.

Bu kısıtlar, mantıksal olarak hatalı veri girişlerini engellemektedir.

### **İş Kuralları (Business Rule) Kısıtları**

Sistemin işleyişine özgü bazı kurallar, veritabanı tasarımında kısıt olarak ele alınmıştır.

- Personel kullanıcılar yalnızca görevli oldukları ilçeye ait konteynerler için rota oluşturabilir.
- Pasif durumda olan kullanıcılar sisteme giriş yapamaz.
- Pasif durumda olan konteynerler rota hesaplamalarına dahil edilmez.
- Bir bildirim yalnızca tek bir konteyner için oluşturulabilir.
- Bir rota, yalnızca bir personel tarafından oluşturulabilir.
- Rota durakları sıralı olmak zorundadır ve aynı rota içerisinde sıra numarası tekrar edemez.

Bu kurallar, uygulama mantığı ile veritabanı yapısının tutarlı çalışmasını sağlamaktadır.

### **Zaman ve Kayıt Tutarlılığı Kısıtları**

Zamana bağlı işlemlerin izlenebilirliğini sağlamak amacıyla ek kısıtlar uygulanmıştır.

- Bildirimler, doluluk geçmişi, rotalar ve eko-puan hareketleri gibi işlem tablolarında tarih ve saat bilgisi zorunludur.
- Sistem kayıtları geriye dönük analiz ve raporlama yapılabilmesine olanak sağlayacak şekilde zaman damgaları ile tutulmaktadır.

### **Genel Değerlendirme**

Bu bölümde tanımlanan kısıtlar, ES-ATIK Akıllı Atık Yönetim Sistemi'nin veri tutarlılığı, güvenliği ve işlevsel doğruluğunu garanti altına almak amacıyla belirlenmiştir. Kısıtlar sayesinde sistem, hem mevcut kullanım senaryolarında hem de ileride yapılabilecek genişletmelerde sağlam ve sürdürülebilir bir veritabanı altyapısına sahip olmaktadır.

### 4.3. G m l  Sistem Tasarımı

ES-ATIK projesinin g m l  sistem katmanı, sistemin sahadaki fiziksel bile enlerini temsil eden ve konteynerlerden ger ek zamanlı veri toplanmasını saėlayan temel altyapıyı olu turmaktadır. Bu katman, yazılım ve kullanıcı aray z  bile enlerinden baėımsız olarak  alı makta olup, sistemin u  noktadaki (edge) karar verme ve veri toplama yeteneklerini barındırmaktadır. G m l  sistem, konteynerlerin doluluk durumunu yerinde algılayarak merkezi sisteme aktarılmasını saėlamakta ve b ylece veri kaynaėına en yakın noktada i lem yapılmasına imk n tanımaktadır.

G m l  sistem tasarımı s recinde temel hedefler; d   k enerji t ketimi ile uzun s reli  alı ma,    mlerde y ksek doėruluk ve tutarlılık, kesintili aė ko ullarında veri kaybının  onlenmesi ve saha ko ullarında minimum bakım gerektiren bir yapı olu turmak olarak belirlenmi tir. Bu hedefler doėrultusunda hem donanım bile enleri hem de g m l  yazılım mimarisi birlikte deėerlendirilmi  ve sistem b t nc l bir bakı  a ısıyla tasarlanmı tır.

Bu kapsamda ESP32 mikrodenetleyicisi tercih edilmi ; d   k g   t ketimi, entegre kablosuz haberle me yetenekleri ve geni  geli tirici desteėi gibi avantajları sayesinde proje gereksinimlerini kar ılayacak bir platform olu turulmu tur. Sens r se imleri ve yazılım mimarisi, ESP32'nin sunduėu bu donanımsal ve yazılımsal imk nlar g    n nde bulundurulularak  ekillendirilmi tir.

#### 4.3.1. Donanım ve Yazılım Ayrı tırması

G m l  sistem tasarımında, sistemin s rd r lebilirliėini ve geli tirilebilirliėini artırmak amacıyla donanım ve yazılım g revleri net bir bi imde ayrı tırılmı tır. Bu yakla ım sayesinde her iki katman kendi sorumluluk alanı i erisinde ele alınmı  ve olası deėi ikliklerin sistem geneline etkisi minimize edilmi tir. Ayrıca bu ayrım, sistemin test s re lerini kolayla tırarak hata ayıklama ve bakım faaliyetlerinin daha verimli y r t lmesine olanak saėlamaktadır.

Donanım Katmanı, fiziksel    mlerin alınmasından ve temel sinyallerin  retilmesinden sorumludur. Bu katmanda ger ekle tirilen ba lıca i lemler  unlardır:

- Ultrasonik sens r aracılıėıyla konteyner i indeki doluluk seviyesinin    lmesi
- Pil voltajının analog giri   zerinden izlenmesi
-    m tetikleme sinyallerinin olu turulması

Yazılım Katmanı (G m l  Yazılım) ise donanımdan alınan ham verilerin i lenmesi ve anlamlı bilgiye d n  t r lmesi g revini  stlenmektedir. Bu kapsamda:

- Sens rden alınan zaman bilgisinin mesafe deėerine d n  t r lmesi
- G r lt l     mlerin median filtre y ntemiyle temizlenmesi
-    m sonu larının y zde doluluk oranına  evrilmesi
- Verilerin JSON formatına d n  t r lerek sunucuya g nderilmesi

Bu ayrıştırma sayesinde donanım tarafında yapılacak sensör değişimi veya devre güncellemesi, yazılımın genel işleyişini etkilemeden gerçekleştirilebilmekte; aynı şekilde yazılım tarafındaki iyileştirmeler donanım müdahalesi gerektirmeden uygulanabilmektedir.

#### 4.3.2. Elektronik Tasarım

Elektronik tasarım aşamasında ESP32 DevKit V1 kartı sistemin merkezi kontrol birimi olarak kullanılmıştır. Ultrasonik ölçümler için HC-SR04 sensörü tercih edilmiş ve konteynerin üst kısmına konumlandırılarak atık seviyesinin doğru ve tekrarlanabilir şekilde ölçülmesi hedeflenmiştir. Bu yerleşim sayesinde sensörün konteyner içindeki atıklardan fiziksel olarak etkilenmesi engellenmiş ve ölçüm doğruluğu artırılmıştır.

Sensör ve kontrol kartı, dış ortam koşullarına dayanıklı olacak şekilde su geçirmez bir muhafaza içerisine yerleştirilmiştir. Bu muhafaza, yağmur, nem, toz ve sıcaklık değişimleri gibi çevresel etkenlere karşı sistemin korunmasını sağlayarak saha kullanımına uygun bir yapı sunmaktadır.

Enerji verimliliği, elektronik tasarımın en kritik kriterlerinden biri olarak ele alınmıştır. Bu doğrultuda:

- Ölçümler belirli zaman aralıklarıyla yapılmış
- Ölçüm dışında kalan zamanlarda ESP32 "Deep Sleep" moduna alınmıştır
- Wi-Fi ve Bluetooth gibi yüksek enerji tüketen bileşenler yalnızca veri gönderimi sırasında aktif edilmiştir

Bu enerji yönetimi stratejileri sayesinde sistemin tek bir pil ile yaklaşık 8 aya kadar kesintisiz çalışabilmesi hedeflenmiş ve saha operasyonları açısından bakım maliyetlerinin düşürülmesi amaçlanmıştır.

#### 4.3.3. Gömülü Yazılım Tasarımı

Gömülü yazılım, Arduino Framework kullanılarak C++ diliyle geliştirilmiştir. Yazılım mimarisi oluşturulurken modülerlik, hata toleransı ve uzun süreli kararlı çalışma ön planda tutulmuştur. Bu sayede sistemin saha koşullarında beklenmeyen durumlara karşı dayanıklı olması sağlanmıştır.

Kablosuz ağ bağlantısının sağlanamadığı durumlarda, ölçülen veriler geçici olarak ESP32'nin dahili **SPIFFS** dosya sistemi üzerinde saklanmaktadır. Ağ bağlantısı tekrar kurulduğunda, biriken bu veriler toplu olarak sunucuya iletilmekte ve veri kaybı yaşanmasının önüne geçilmektedir.

Ayrıca yazılım, sensör arızaları veya olağan dışı ölçüm değerlerini tespit edebilecek kontroller içermektedir. Belirlenen eşiklerin dışında kalan veya fiziksel olarak mümkün olmayan ölçümler sistem tarafından filtrelenmekte ve geçersiz veri olarak işaretlenmektedir. Bu sayede merkezi sisteme hatalı veri aktarımı engellenmekte ve veri kalitesi korunmaktadır.

Bu yapı sayesinde ES-ATIK projesinin gömülü sistem katmanı, sahada uzun süre insan müdahalesi gerektirmeden, güvenilir ve sürdürülebilir bir şekilde çalışabilen bir çözüm sunmaktadır.

#### 4.4. Yazılım Tasarımı

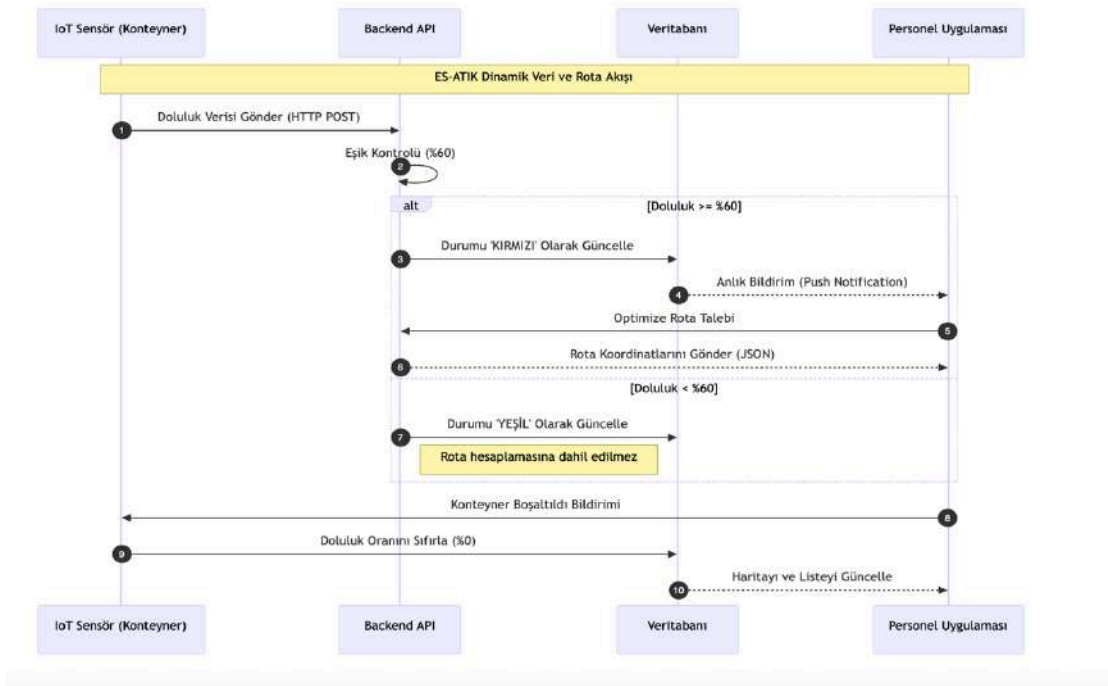
Sistemin yazılım altyapısı ölçeklenebilir ve modüler bir yapıda kurgulanmıştır.

##### 4.4.1. Gereksinime Bağlı Tasarım Kalıp Seçimi

Sistemde **Observer (Gözlemci) Tasarım Kalıbı** kullanılmıştır. Bu tasarım yaklaşımı sayesinde, veritabanında yer alan konteynerlerin doluluk oranları sürekli olarak izlenmektedir. Herhangi bir konteynerin doluluk oranı belirlenen eşik değeri olan %60 seviyesini aştığında, bu durum sistem tarafından otomatik olarak bir *olay (event)* olarak algılanır. İlgili konteyneri izleyen (observer rolündeki) personel uygulamaları ve rota yönetim modülü bu değişiklikten anlık olarak haberdar edilir.

Bu mekanizma sayesinde manuel kontrol ihtiyacı ortadan kaldırılırken, toplama rotaları doluluk durumuna göre dinamik ve otomatik biçimde güncellenmektedir. Böylece hem zaman kaybı önlenmekte hem de gereksiz rota ziyaretlerinin önüne geçilmektedir. Observer tasarım kalıbının sağladığı bu gevşek bağlı (loosely coupled) yapı, sistemin ölçeklenebilirliğini artırmakta ve ileride yeni bildirim türlerinin veya farklı kullanıcı rollerinin sisteme kolayca entegre edilebilmesine olanak tanımaktadır.

##### 4.4.2. UML Kullanarak Tasarım Diyagramları Oluşturma



Şekil 4.2: ES-ATIK Sistemi Uçtan Uca Dinamik Etkileşim (Sequence) Diyagramı

Yukarıdaki ardışıl (sequence) diyagramı, projenin donanım katmanı (IoT sensör), servis katmanı (Backend API) ve kullanıcı katmanı (Mobil Uygulama) arasındaki veri trafiğini ve zaman tabanlı etkileşimi modellemektedir.

Süreç şu şekilde işlemektedir:

1. **Veri Toplama:** IoT sensör birimi, konteyner doluluk oranını ölçerek HTTP protokolü üzerinden JSON formatında sunucuya iletir.
2. **Karar Mekanizması:** Backend birimi gelen veriyi %60 eşik değerine göre analiz eder.
3. **Bildirim ve Optimizasyon:** Kritik doluluğa ulaşan konteynerler için veritabanında durum güncellemesi yapılır ve bu durum şoför uygulamasına asenkron bir 'Push Notification' olarak yansır.
4. **Dinamik Rota:** Şoför uygulaması, Google Maps API entegrasyonu ile sadece doluluğu onaylanmış konteynerleri içeren en kısa rotayı kullanıcıya sunar.

Bu yapı, sistemin insan müdahalesine gerek kalmadan, veriye dayalı ve olay güdümlü (event-driven) bir şekilde çalışmasını garanti altına almaktadır.

Sistemin mantıksal işleyişi Sequence (Ardışıl) ve Class (Sınıf) diyagramları ile modellenmiştir. Sequence diyagramında; sensörden verinin çıkışı, Firebase üzerinden işlenişi ve personelin ekranına düşene kadar geçen tüm süreçler dikey zaman çizelgesinde detaylandırılmıştır.

#### 4.5. Test Tasarımı

Sistem geliştirme yaşam döngüsünün kritik bir aşaması olan test tasarımı, ES-ATIK projesinin hem donanım kararlılığını hem de yazılım doğruluğunu garanti altına almak amacıyla yapılandırılmıştır. Yazılım ve donanım bileşenleri, sahaya sürülmeden önce V-Model prensiplerine uygun kapsamlı bir test planına tabi tutulmuştur.

##### 4.5.1. Gereksinim Analizlerinden Test Hedeflerinin Belirlenmesi

Test stratejisinin temel odağı, analiz aşamasında belirlenen işlevsel ve teknik gereksinimlerin sayısal olarak doğrulanmasıdır.

- **Hassasiyet Hedefi:** IoT sensör biriminin ölçüm hata payının  $\pm\%3$  eşiklerinin altında kalması hedeflenmiştir.
- **Performans Hedefi:** Dinamik rota hesaplama ve optimizasyon algoritmasının yanıt süresinin 2 saniyeyi geçmemesi temel kriter olarak belirlenmiştir.

##### 4.5.2. Birim (Unit) Testleri Tasarımı

ES-ATIK sisteminin en küçük yapı taşlarını oluşturan fonksiyonların ve iş kurallarının tekil doğruluğu, Hafta 11 ders notlarında belirtilen birim (unit) test prensipleri doğrultusunda analiz edilmiştir. Bu testler, her bir modülün sistemin diğer bileşenlerinden izole edilerek, dış veri kaynaklarından bağımsız bir şekilde mantıksal olarak doğru sonuçlar üretip üretmediğini denetlemek amacıyla tasarlanmıştır.



### T01 – Doluluk Yüzdesi Hesaplama Fonksiyonu Testi

- **TEST TANIMI:** Unit\_Calculate\_Fill\_Level\_Logic
- **İLGİLİ MODÜL:** Gömülü Yazılım Katmanı / Veri İşleme ve Dönüştürme Birimi
- **TESTİN AMACI:** Bu testin temel amacı, IoT uç birimindeki ultrasonik sensörden (HC-SR04) elde edilen ve ham "uçuş süresi" verisinden türetilen mesafe bilgisinin (santimetre cinsinden), konteynerin önceden tanımlanmış fiziksel derinlik parametrelerine göre yüzdesel doluluk oranına dönüştürülmesindeki matematiksel tutarlılığı doğrulamaktır. Fonksiyonun sadece lineer hesaplama kapasitesi değil, aynı zamanda fiziksel sınır değerlerde (tam dolu/tam boş) ve beklenmeyen uç değerlerde (limit aşımı) nasıl tepki verdiği, sistemin veri doğruluğunu garanti altına almak amacıyla analiz edilir.
- **ÖN KOŞULLAR:** Sistemin fiziksel konteyner derinlik sabitinin (Referans: 80 cm) bellek üzerinde tanımlanmış ve erişilebilir olması; matematiksel hesaplama kütüphanelerinin sisteme entegre olması gerekir.
- **TEST VERİ SETİ:**
  - **Tam Dolu Sınırı (Lower Bound):** Girdi: 0 cm (Sensörün konteyner ağzına kadar dolu olduğunu simüle eder).
  - **Boş Konteyner Sınırı (Upper Bound):** Girdi: 80 cm (Sensörün zemini gördüğü, tamamen boş konteyner durumu).
  - **Doğrusal Ara Değer 1:** Girdi: 20 cm (Konteynerin dörtte üçünün dolu olduğunu simüle eder).
  - **Doğrusal Ara Değer 2:** Girdi: 40 cm (Konteynerin yarısının dolu olduğunu simüle eder).
  - **Hatalı/Limit Dışı Değerler:** Girdi: -5 cm (Sinyal gürültüsü durumu) ve 120 cm (Fiziksel derinliği aşan hatalı okuma).
- **BEKLENEN ÇIKTI:** Girilen santimetre verisiyle ters orantılı, 0 ile 100 aralığında matematiksel olarak doğrulanmış tam sayı veya ondalıklı doluluk yüzdesi.
- **BAŞARI KRİTERİ:** Fonksiyonun tüm rasyonel aralıklarda matematiksel hatasız çalışması (Örn: 20 cm girişi için tam %75 üretmesi) ve 80 cm üzerindeki değerler için %0 sonucunu kararlı şekilde döndürmesi.

### T02 – Doluluk Eşik Değeri Karar Fonksiyonu Testi

- **TEST TANIMI:** Unit\_Threshold\_Decision\_Logic
- **İLGİLİ MODÜL:** Karar Destek Algoritması / Otonom Rota Tetikleme Birimi
- **TESTİN AMACI:** ES-ATİK sisteminin operasyonel verimliliğinin merkezinde yer alan %60 doluluk eşik kuralının, yazılımsal olarak doğru kararları üretip üretmediği denetlenir. Bu fonksiyon, vatandaş arayüzündeki pin renginin yeşilden kırmızıya dönmesini ve şoför paneline yeni bir görev atanmasını tetikleyen temel mantıksal birimdir. Testin amacı, sistemin tam sınır değerde (boundary value) nasıl davrandığını, kritik eşğin bir altındaki (%59) veya bir üzerindeki (%61) değerlerde yanlış karar üretip üretmediğini doğrulamaktır.
- **ÖN KOŞULLAR:** Sistemin konfigürasyon dosyasında "THRESHOLD\_LEVEL" değerinin 60 olarak tanımlanmış olması gerekmektedir.
- **TEST VERİ SETİ:**
  - **Eşik Altı Güvenli Bölge:** Girdi: %0, %30 (Herhangi bir aksiyon gerekmez).
  - **Kritik Karar Sınırı (Alt):** Girdi: %59 (Eşğin hemen altı, rota tetiklenmemelidir).
  - **Eşik Sınır Noktası:** Girdi: %60 (Tam eşik değeri, otonom süreç başlamalıdır).

- **Eşik Üstü Pozitif Bölge:** Girdi: %61, %85, %100 (Eşik aşıldı, sistem tetiklenmelidir).
- **BEKLENEN ÇIKTI:** Mantıksal Boolean çıktı (True: Rota Ekle/Kırmızı Statü, False: Eylem Gereksiz/Yeşil Statü).
- **BAŞARI KRİTERİ:** Fonksiyonun %60 ve üzerindeki tüm değerlerde istisnasız "True", %60'ın altındaki değerlerde ise "False" döndürerek karar bütünlüğünü %100 doğrulukla koruması.

### T03 – Bildirim Oluşturma Parametre Doğrulama Testi

- **TEST TANIMI:** Unit\_Notification\_Input\_Validation
- **İLGİLİ MODÜL:** Vatandaş Etkileşim Katmanı / Veri Doğrulama (Validation) Modülü
- **TESTİN AMACI:** Vatandaşlar tarafından gönderilen bildirimlerin (taşma, hasar vb.) MariaDB veritabanına kaydedilmeden önce veri bütünlüğü filtresinden geçirilmesini denetler. Testin amacı, zorunlu alanların (Konteyner\_ID, Kullanıcı\_ID, Bildirim\_Tipi) eksik veya hatalı olması durumunda sistemin veritabanında boş/hatalı kayıt oluşturmasını engelleme yeteneğini ölçmektir. Bu test, veritabanı kısıtlarının (Constraints) uygulama seviyesinde korunmasını sağlar.
- **ÖN KOŞULLAR:** Kullanıcı oturumunun doğrulanmış olması ve sistemin validasyon kütüphanelerinin aktif olması gerekir.
- **TEST VERİ SETİ :**
  - **Eksik Konteyner Kimliği:** Girdi: {konteyner\_id: null, kullanıcı\_id: 15, tip: "taşma"}.
  - **Eksik Kullanıcı Kimliği:** Girdi: {konteyner\_id: 101, kullanıcı\_id: null, tip: "hasar"}.
  - **Hatalı Veri Tipi:** Girdi: {konteyner\_id: "ABC", kullanıcı\_id: 15, tip: 123} (ID sayı, tip metin olmalı).
  - **Geçerli Veri Seti:** Girdi: {konteyner\_id: 101, kullanıcı\_id: 15, tip: "koku"}.
- **BEKLENEN ÇIKTI:** Eksik/Hatalı verilerde "Validation\_Error" mesajı ve işlem reddi; tam verilerde "Success" onayı.
- **BAŞARI KRİTERİ:** Fonksiyonun eksik parametre içeren hiçbir bildirimi MariaDB'ye ulaştırmadan başarıyla reddetmesi ve sistem güvenliğini sağlaması.

### T04 – Eko-Puan Artış Hesaplama Fonksiyonu Testi

- **TEST TANIMI:** Unit\_EcoPoint\_Aggregation\_Logic
- **İLGİLİ MODÜL:** Teşvik Sistemi / Kullanıcı Sadakat ve Oyunlaştırma Modülü
- **TESTİN AMACI:** Sistemin sosyal sorumluluk ayağını yöneten puanlama algoritmasının matematiksel doğruluğunu hedefler. Amaç, onaylanan her bildirim sonrasında kullanıcının puanının MariaDB'deki profil tablosunda doğru miktarda (+5 puan) ve birikimli (cumulative) olarak artırılmasını sağlamaktır. Puanın matematiksel olarak doğru artması, kullanıcı motivasyonu ve veritabanı tutarlılığı için kritik öneme sahiptir.
- **ÖN KOŞULLAR:** Kullanıcı puan bilgisinin veritabanından başarıyla okunabilmesi ve "POINT\_STEP" değerinin 5 olarak tanımlanmış olması.
- **TEST VERİ SETİ:**
  - **Başlangıç Senaryosu:** Mevcut Puan: 0, İşlem: +1 Onay (Sonuç 5 olmalı).
  - **Birikimli Artış:** Mevcut Puan: 145, İşlem: +1 Onay (Sonuç 150 olmalı).
  - **Arka Arkaya İşlem:** Mevcut Puan: 150, Peş peşe 3 onay (Sonuç 165 olmalı).

- **Negatif Giriş Kontrolü:** Artış miktarı olarak -50 verilmesi (Sistem tarafından reddedilmelidir).
- **BEKLENEN ÇIKTI:** Güncellenmiş yeni toplam puan değeri (Integer).
- **BAŞARI KRİTERİ:** Fonksiyonun her geçerli işlemde puanı tam olarak belirlenen katsayı kadar artırması ve veri tipini (Integer) bozmadan saklaması.

#### T05 – JSON Veri Yapısı Doğrulama Fonksiyonu Testi

- **TEST TANIMI:** Unit\_IoT\_JSON\_Structure\_Check
- **İLGİLİ MODÜL:** Haberleşme Katmanı / Veri Ayırıştırma (Parser) ve Güvenlik Birimi
- **TESTİN AMACI:** IoT cihazından MariaDB bulut sunucusuna gönderilen veri paketlerinin, önceden tanımlanmış JSON şemasına uygunluğunu denetler. Testin amacı, sistemin sadece tanıdığı anahtarları (Key) ve uygun veri tiplerini kabul ederek, hatalı veri girişi veya SQL injection gibi siber güvenlik risklerini bertaraf etmektir. Bu fonksiyon, sistemin veri girişindeki en güçlü güvenlik duvarıdır.
- **ÖN KOŞULLAR:** Sistemde geçerli JSON şemasının (device\_id: int, fill: int, battery: int) tanımlı olması.
- **TEST VERİ SETİ:**
  - **Geçerli Standart Paket:** Girdi: {"device\_id": 55, "fill\_level": 70, "battery": 3700}.
  - **Eksik Anahtar (Key) Testi:** Girdi: {"fill\_level": 70, "battery": 3700} (device\_id eksik).
  - **Hatalı Veri Tipi Testi:** Girdi: {"device\_id": 55, "fill\_level": "Yüksek"} (Integer yerine String).
  - **Zararlı Veri Enjeksiyonu:** Girdi: {"device\_id": 55, "cmd": "DROP TABLE"}.
- **BEKLENEN ÇIKTI:** Valid (Geçerli) veya Invalid (Geçersiz) durum sonucu.
- **BAŞARI KRİTERİ:** Sadece şemaya tam uyumlu paketlerin kabul edilmesi, geri kalan tüm bozuk veya şüpheli paketlerin loglanarak reddedilmesi.

#### T06 – Personel Yetki Kontrol Fonksiyonu Testi

- **TEST TANIMI:** Unit\_Personnel\_Access\_Rights\_Control
- **İLGİLİ MODÜL:** Güvenlik ve Yetkilendirme (AuthZ) Katmanı / Personel Yönetim Birimi
- **TESTİN AMACI:** Projenin operasyonel güvenliğini sağlamak amacıyla, sadece yetkili ve statüsü "Aktif" olan personelin rota oluşturabilmesini veya durum güncellemesi yapabilmesini denetler. Bu testin amacı, rol tabanlı erişim kontrolünün (RBAC) doğruluğunu ve bir personelin statüsü değiştiğinde (Örn: Pasife alınma) sistem yetkilerinin anında kısıtlanıp kısıtlanmadığını doğrulamaktır.
- **ÖN KOŞULLAR:** MariaDB veritabanında "User\_Role" ve "User\_Status" alanlarının güncel olarak tanımlanmış olması.
- **TEST VERİ SETİ:**
  - **Tam Yetkili Senaryo:** Profil: {rol: "Sofor", durum: "Aktif"} (Onaylanmalı).
  - **Yetkisiz Rol Senaryosu:** Profil: {rol: "Vatandas", durum: "Aktif"} (Reddedilmeli).
  - **Yetkili Ama Pasif Senaryo:** Profil: {rol: "Sofor", durum: "Pasif"} (Reddedilmeli).
  - **Tanımsız Kullanıcı:** Profil: {rol: null, durum: null} (Erişim engellenmeli).
- **BEKLENEN ÇIKTI:** Erişim İzni (Granted) veya Erişim Reddi (Denied).

- **BAŞARI KRİTERİ:** Fonksiyonun hem rolü hem de personelin aktiflik durumunu aynı anda kontrol ederek sadece "Uygun" statüsündeki şoförlere işlem izni vermesi.

#### T07 – Kullanıcı Kimlik Doğrulama (Giriş Yap) Fonksiyonu Testi

- **TEST TANIMI:** Unit\_User\_Authentication\_Check
- **İLGİLİ MODÜL:** Kullanıcı Kimlik Doğrulama (Authentication) / Güvenlik Modülü
- **TESTİN AMACI:** Kullanıcının girdiği kullanıcı adı ve şifre bilgilerinin MariaDB veritabanındaki kayıtlarla eşleşip eşleşmediğini ve seçilen "Rol" (Vatandaş/Personel) bilgisine göre doğru yetkilendirme yapıp yapılmadığını denetler.
- **ÖN KOŞULLAR:** Kullanıcının sistemde önceden kayıtlı bir hesabı bulunmalıdır.
- **TEST VERİ SETİ:**
  - **Geçerli Giriş:** Doğru kullanıcı adı + Doğru şifre + Doğru rol seçimi.
  - **Hatalı Şifre:** Doğru kullanıcı adı + Yanlış şifre.
  - **Eksik Bilgi:** Kullanıcı adı dolu + Şifre boş.
  - **Rol Uyuşmazlığı:** Şoför hesabı bilgileriyle "Vatandaş" rolü seçilerek giriş denemesi.
- **BEKLENEN ÇIKTI:** Geçerli bilgilerde "Giriş Başarılı" yanıtı ve ana panele yönlendirme; hatalı veya eksik bilgilerde "Hatalı Kullanıcı Adı veya Şifre" uyarısı.
- **BAŞARI KRİTERİ:** Sistemin sadece veritabanı ile %100 eşleşen bilgilerle girişe izin vermesi ve rol ayırımını doğru yapması.

#### T08 – Yeni Kullanıcı Kayıt (Kayıt Ol) Validasyon Testi

- **TEST TANIMI:** Unit\_User\_Registration\_Validation
- **İLGİLİ MODÜL:** Vatandaş Kayıt Modülü / Veri Giriş Katmanı
- **TESTİN AMACI:** Yeni kayıt formundaki zorunlu alanların (Ad Soyad, TC Kimlik No, Mahalle, Şifre) doğruluğunu ve format uygunluğunu denetler. Özellikle TC Kimlik No alanının 11 haneli sayısal veri kuralına uyup uymadığı kontrol edilir.
- **ÖN KOŞULLAR:** Veritabanı bağlantısının aktif olması ve "Kayıt Ol" sayfasının açık olması.
- **TEST VERİ SETİ:**
  - **Eksik Alan:** Ad-Soyad dolu, TC Kimlik No boş bırakılarak kayıt denemesi.
  - **Hatalı Format:** TC Kimlik No alanına 11 haneden az sayı veya harf girişi.
  - **Mahalle Seçimi:** Açılır menüden (Dropdown) mahalle seçilmemesi durumu.
  - **Şifre Güvenliği:** 3 karakterli (çok kısa) şifre denemesi.
- **BEKLENEN ÇIKTI:** Hatalı veya eksik alanlarda ilgili input altında kırmızı uyarı metinlerinin belirmesi; tüm alanlar doğruysa "Kayıt Başarıyla Tamamlandı" mesajı.
- **BAŞARI KRİTERİ:** Formun sadece belirlenen iş kurallarına (Zorunlu alanlar, 11 haneli TC vb.) tam uyum sağlandığında veritabanına kayıt isteği (POST) göndermesi.

#### 4.5.3. Entegrasyon Testleri Tasarımı

Entegrasyon testleri, ES-ATIK sistemini oluşturan alt modüllerin (IoT Birimi, MariaDB Veritabanı, API Servisleri ve Flutter Mobil Arayüzleri) birbirleriyle olan etkileşimini ve veri alışverişi süreçlerini doğrulamak amacıyla kurgulanmıştır. Hafta 11 ders notlarında vurgulanan "Modüller Arası İletişim Hatası" riskini minimize etmek için bu aşamada verinin bir uçtan diğer uca (End-to-End) kayıpsız ve senkronize bir şekilde iletildiği denetlenmektedir.

**IT01 – Vatandaş Bildirimi → Veritabanı Kaydı → "Bildirimlerim" Senkronizasyonu**

- **TEST TANIMI:** Integration\_Citizen\_Report\_Sync
- **İLGİLİ MODÜLLER:** Vatandaş Mobil Arayüz Modülü, REST API Servis Katmanı, MariaDB Veritabanı (TBL\_BILDIRIM).
- **TESTİN AMACI:** Bu entegrasyon testinin temel amacı, vatandaşın mobil uygulama arayüzünden tetiklediği bir "atık bildirimi" eyleminin sistemdeki tüm katmanlardan geçişini doğrulamaktır. Verinin kullanıcı arayüzünden (UI) JSON formatında paketlenmesi, API üzerinden MariaDB veritabanına kalıcı olarak işlenmesi ve ardından aynı oturum içerisinde veritabanından tekrar sorgulanarak "Bildirimlerim" sekmesinde hatasız bir şekilde listelenmesi sürecindeki veri tutarlılığı analiz edilir. Bu süreçte asenkron veri iletiminde yaşanabilecek gecikmelerin (latency) veya veri tipi uyumsuzluklarının (mapping errors) tespiti hedeflenmektedir.
- **ÖN KOŞULLAR:** \* Sisteme geçerli bir "Vatandaş" hesabı ile giriş yapılmış olması.
  - Mobil cihazın aktif bir internet bağlantısına ve GPS verisine sahip olması.
  - MariaDB veritabanı servislerinin "Erişilebilir" statüsünde bulunması.
- **TEST VERİ SETİ:** \* **Hedef Konteyner ID:** CONT-204-ODP
  - **Bildirim Kategorisi:** Dolu/Taşma veya Bozuk/Hasarlı
  - **Ek Veri:** Kullanıcı açıklaması (Opsiyonel metin girişi).
  - **Konum Verisi:** Enlem: 39.7500, Boylam: 30.4800 (Eskişehir yerel koordinatları).
  - **Zaman Damgası:** YYYY-MM-DD HH:mm:ss (Sistem tarafından otomatik üretilen veri).
- **BEKLENEN ÇIKTI:** API katmanından dönen "201 Created" yanıtı ve ardından "Bildirimlerim" ekranında ilgili konteyner kimliği ile bildirim türünün (Örn: "Konteyner 204 - Taşma Bildirimi") liste elemanı olarak görüntülenmesi.
- **BAŞARI KRİTERİ:** Veritabanındaki TBL\_BILDIRIM tablosuna eklenen satır sayısı ile mobil arayüzdeki liste eleman sayısının %100 örtüşmesi ve verinin görselleştirme sırasında bozulmaması.

#### IT02 – Bildirim → Eko-Puan Güncellemesi → Header Senkronizasyonu

- **TEST TANIMI:** Integration\_Report\_EcoPoint\_Header\_Update
- **İLGİLİ MODÜLLER:** Bildirim Yönetim Modülü, Puanlama Algoritması, Uygulama Global State (Header) Katmanı.
- **TESTİN AMACI:** Sistemin oyunlaştırma (gamification) ayağını oluşturan Eko-Puan mekanizmasının modüller arası çalışma başarısını denetler. Bir vatandaş bildirim gönderdiğinde, arka plandaki puanlama servisinin tetiklenmesi, kullanıcının MariaDB'deki puan bakiyesinin güncellenmesi ve bu değişikliğin uygulamanın tüm sayfalarında üst bilgi (Header) alanında anlık (real-time) olarak güncellenmesi süreci test edilir. Bu test, özellikle "State Management" (Durum Yönetimi) mekanizmasının veritabanı ile ne kadar hızlı senkronize olduğunu doğrulamak için tasarlanmıştır.
- **ÖN KOŞULLAR:** \* Kullanıcının başlangıç puanının (Örn: 100) veritabanından doğru okunmuş olması.
  - Puan artış katsayısının sistemde tanımlı olması.
- **TEST VERİ SETİ (DETAYLICA):** \* **Başlangıç Durumu:** Kullanıcı Eko-Puanı = 100.
  - **Tetikleyici Eylem:** Geçerli bir konteyner bildiriminin sunucuya iletilmesi.
  - **Puan Artış Katsayısı:** +5 birim.
  - **UI Toast Mesajı:** "Tebrikler! +5 Eko-Puan Kazandınız" (Sistem uyarısı).
- **BEKLENEN ÇIKTI:** Bildirim sonrası Header bileşenindeki metin alanının anlık olarak "105" değerine evrilmesi.

- **BAŞARI KRİTERİ:** Veritabanındaki puan sütunu ile Header'daki görsel verinin senkron olması ve kullanıcı sayfalar arası geçiş yaptığında (Refresh) puanın doğru kalmaya devam etmesi.

### IT03 – Personel: Kırmızı Konteynerler → Rota Oluştur → Harita Polyline

- **TEST TANIMI:** Integration\_Route\_Optimization\_Visualizer
- **İLGİLİ MODÜLLER:** MariaDB Konteyner Veri Katmanı, Rota Optimizasyon Algoritması, Google Maps API / Polyline Çizim Modülü.
- **TESTİN AMACI:** Bu test, sistemin operasyonel katmanındaki en karmaşık entegrasyon sürecini ele alır. Veritabanındaki doluluk oranı %60'ın üzerinde olan konteynerlerin (Kırmızı Statü) doğru bir şekilde filtrelenmesi, bu koordinatların rota algoritmasına girdi olarak beslenmesi ve algoritmanın ürettiği "en kısa yol" çözümünün harita üzerinde bir "Polyline" (Güzergah çizgisi) olarak görselleştirilmesini doğrular. Bu aşamada veritabanı, matematiksel mantık ve coğrafi bilgi sistemi (GIS) servislerinin tam uyum içinde çalışması denetlenmektedir.
- **ÖN KOŞULLAR:** \* Veritabanında en az 3 adet doluluk oranı %60'ın üzerinde konteynerin mevcut olması.
  - Personel (Şoför) hesabının sisteme giriş yapmış olması.
  - Google Maps API anahtarının aktif ve yetkili olması.
- **TEST VERİ SETİ:** \* **Girdi Verisi:** Doluluk  $\geq 60$  filtresiyle dönen koordinat listesi (Örn: K1, K4, K7).
  - **Başlangıç Noktası:** Şoförün mevcut GPS konumu.
  - **Algoritma Parametresi:** "En Kısa Mesafe Öncelikli" (Dijkstra veya Greedy Search).
  - **Harita Bileşeni:** GoogleMap() widget'ı içerisindeki PolylineSet.
- **BEKLENEN ÇIKTI:** Şoför ekranında sıralı bir durak listesi panelinin açılması ve harita üzerinde bu durakları birbirine bağlayan kesintisiz bir rota çizgisinin belirmesi.
- **BAŞARI KRİTERİ:** Algoritmanın ürettiği durak sıralaması ile haritadaki görsel güzergahın birebir aynı olması ve tüm kırmızı konteynerlerin rotaya dahil edilmiş olması.

### IT04 – Personel: “Toplandı İşaretle” → Konteyner Durumu → Rota Durak Güncellemesi

- **TEST TANIMI:** Integration\_Collection\_Update\_Workflow
- **İLGİLİ MODÜLLER:** Personel Mobil Arayüzü, MariaDB Update Prosedürleri, Rota Takip/State Yönetimi.
- **TESTİN AMACI:** Saha personelinin gerçekleştirdiği fiziksel toplama işleminin dijital sisteme geri beslenmesi sürecini denetler. Şoför bir durakta "Toplandı" butonuna bastığında; işlemin veritabanındaki konteyner doluluğunu sıfırlaması, aynı zamanda aktif rota listesinden o durağın statüsünü "Tamamlandı" olarak değiştirmesi ve navigasyon akışını bir sonraki durağa yönlendirmesi arasındaki entegre iş akışı doğrulanır. Bu test, operasyonel veri döngüsünün (Feedback Loop) kapandığı kritik noktadır.
- **ÖN KOŞULLAR:** \* En az bir durak içeren "Aktif" statülü bir rotanın şoför panelinde açık olması.
  - Konteyner ID'sinin rota durak listesiyle eşleşiyor olması.
- **TEST VERİ SETİ:** \* **Hedef Durak:** Konteyner ID: 45
  - **Mevcut Doluluk:** %85 (İşlem öncesi).
  - **Eylem:** onPressed: markAsCollected() (Arayüz tetikleyicisi).

- **Veritabanı Güncelleme:** UPDATE containers SET fill\_level = 0 WHERE id = 45.
- **BEKLENEN ÇIKTI:** Rota listesindeki 45 numaralı durağın yanına "Check" (Tamamlandı) işareti gelmesi ve haritadaki pin renginin kırmızıdan yeşile dönmesi.
- **BAŞARI KRİTERİ:** Tek bir "Toplandı" komutunun hem TBL\_KONTEYNER hem de TBL\_ROTA tablolarında eşzamanlı ve tutarlı değişiklik yapması.

#### 4.5.4. Sistem Testleri (Fonksiyonel ve Performans) Tasarımı

Sistem testleri, ES-ATIK projesinin tüm alt bileşenlerinin entegre bir şekilde, nihai kullanıcı senaryoları altında ve gerçek dünya koşullarında nasıl performans gösterdiğini denetleyen en kapsamlı test aşamasıdır. Hafta 11 ders notlarında belirtilen "Uçtan Uca (End-to-End) Test" prensibi doğrultusunda, bu bölümde sadece tekil fonksiyonlar değil; sistemin rol bazlı güvenlik mekanizmaları (fonksiyonel) ve yüksek veri yükü altındaki tepki süreleri (performans) analiz edilmektedir. Bu aşama, projenin en başında tanımlanan "İşlevsel" ve "İşlevsel Olmayan" gereksinimlerin karşılandığının son onayı niteliğindedir.

##### ST01 – Rol Bazlı Yönlendirme ve Erişim Testi (Fonksiyonel)

- **TEST TANIMI:** System\_Role\_Based\_Access\_Control\_Auth
- **İLGİLİ MODÜL:** Güvenlik Modülü / Yönlendirme (Routing) ve Yetkilendirme (AuthZ) Katmanı
- **TESTİN AMACI:** Sistemin rol tabanlı erişim kontrolü (RBAC) mekanizmasının, kullanıcının kimlik doğrulama (Authentication) sonrası sahip olduğu yetki düzeyine göre doğru yönetim paneline (Vatandaş veya Personel) yönlendirilip yönlendirilmediğini doğrulamaktır. Ayrıca, "Yatay Yetki Aşımı" riskini test etmek amacıyla; düşük yetkili bir kullanıcının (Vatandaş), yüksek yetkili bir arayüze (Personel Rotası) doğrudan URL veya uygulama içi derin linkler üzerinden erişmeye çalışması durumunda sistemin sergilediği güvenlik bariyeri analiz edilir.
- **ÖN KOŞULLAR:** \* Veritabanında aktif "Vatandaş" ve "Personel/Şoför" hesaplarının tanımlı olması.
  - Firebase Authentication servisinin aktif ve erişilebilir durumda olması.
- **TEST VERİ SETİ:** \* **Hesap A (Vatandaş):** vatandas@esatik.com / pass1234
  - **Hesap B (Personel):** sofor@esatik.com / sofor5678
  - **Hedef Güvenli Rota:** /personel-dashboard/route-optimizer (Vatandaşın erişmemesi gereken path).
- **BEKLENEN ÇIKTI:** Hesap tipine göre otomatik panel yönlendirmesi; yetkisiz erişim denemesinde "403 Forbidden" uyarısı veya ana sayfaya otomatik geri yönlendirme.
- **BAŞARI KRİTERİ:** Sistemin rol ayrımını %100 doğrulukla yapması ve yetkisiz hiçbir isteğin personel paneli bileşenlerini render etmesine izin vermemesi.

##### ST02 – En Yakın 5 Konteyner Akışı Testi (Fonksiyonel)

- **TEST TANIMI:** System\_Nearest\_Container\_Flow\_Validation
- **İLGİLİ MODÜL:** Konum Servisleri, MariaDB Coğrafi Sorgu Birimi, Harita Görselleştirme Modülü.
- **TESTİN AMACI:** Sistemin vatandaşlara sunduğu en temel değer önerisi olan "en yakın konteyner tespiti" sürecinin uçtan uca doğruluğunu denetler. Kullanıcının GPS koordinatlarının alınmasından başlayarak, MariaDB üzerinde gerçekleştirilen

"Haversine" veya "Euclidean" mesafe sorgusunun sonuçlarının liste görünümü ile harita üzerindeki marker (işaretçi) pozisyonları arasındaki tutarlılığı analiz edilir. Bu test, veritabanından çekilen "Top 5" kaydın görsel arayüzde doğru senkronizasyonla ve veri kaybı olmadan sunulmasını garanti altına alır.

- **ÖN KOŞULLAR:** \* Cihazın konum (GPS) izinlerinin verilmiş olması.
  - Veritabanında Eskişehir genelinde konumlandırılmış en az 10 adet konteyner verisinin bulunması.
- **TEST VERİ SETİ:** \* **Simüle Edilen Konum:** Enlem: 39.7490, Boylam: 30.4780 (Büyükdere Mahallesi merkezi).
  - **Veritabanı Kayıtları:** Çeşitli mesafelerde (100m, 300m, 500m, 1km, 5km) tanımlanmış konteyner koordinatları.
  - **Görsel Filtre:** "Sadece En Yakın 5 Kayıt" parametresi.
- **BEKLENEN ÇIKTI:** Harita üzerinde kullanıcının etrafındaki 5 adet marker'ın belirmesi ve bu marker'lara ait bilgilerin (mesafe, doluluk vb.) alttaki liste paneliyle birebir eşleşmesi.
- **BAŞARI KRİTERİ:** Listenin mesafe bazlı küçükten büyüğe sıralanması, 6. en yakın konteynerin kesinlikle gösterilmemesi ve harita-liste senkronizasyonunun tam olması.

#### ST03 – Harita Yükleme Süresi ve Marker Render Testi (Performans)

- **TEST TANIMI:** System\_Map\_Rendering\_Performance\_Latency
- **İLGİLİ MODÜL:** Ön Yüz (Frontend) Render Motoru, Google Maps API Entegrasyonu.
- **TESTİN AMACI:** Hafta 11 ders notlarında vurgulanan "Fonksiyonel Olmayan Gereksinimler" kapsamında, sistemin kullanıcı deneyimini doğrudan etkileyen hız parametrelerini ölçer. Harita bileşeninin ilk yükleme süresi (Initial Load Time) ve veritabanından gelen dinamik konteyner verilerinin marker olarak haritaya işleme süresi (Marker Rendering Time) analiz edilir. Bu testin amacı, internet trafiğinin yoğun olduğu veya veritabanı sorgu yükünün arttığı anlarda bile uygulamanın "donma" yaşamadan akıcı bir deneyim sunup sunmadığını saptamaktır.
- **ÖN KOŞULLAR:** \* Standart mobil veri hızı (Örn: 10 Mbps 4G bağlantısı).
  - Ön belleğin (Cache) temizlenmiş olması (Soğuk başlangıç testi için).
- **TEST VERİ SETİ:** \* **Veri Yüğü:** 10 adet eş zamanlı marker verisi.
  - **Bağlantı Türü:** WiFi ve 4G olmak üzere iki farklı ağ senaryosu.
  - **Hedef Eşik Değer:** < 3.0 Saniye (Hafta 9 UX standartlarına göre).
- **BEKLENEN ÇIKTI:** Uygulama açılışından haritanın interaktif hale gelmesine kadar geçen milisaniye (ms) cinsinden süre ölçümü.
- **BAŞARI KRİTERİ:** Toplam yükleme süresinin 3 saniyenin altında kalması ve marker'ların harita üzerinde titreme (flicker) yapmadan stabil şekilde render edilmesi.

#### ST04 – Rota Oluşturma Süresi Testi (Performans)

- **TEST TANIMI:** System\_Route\_Optimization\_Calculation\_Speed
- **İLGİLİ MODÜL:** Arka Plan (Backend) Rota Algoritması, MariaDB Query Optimizer.
- **TESTİN AMACI:** Personel panelinde "Rota Oluştur" komutu verildiğinde, sistemin karmaşık matematiksel hesaplamaları (TSP - Traveling Salesman Problem benzeri) ne kadar sürede sonuçlandırdığını denetler. Özellikle doluluk oranı %60'ı geçmiş "Kırmızı" statüdeki konteyner sayısının arttığı senaryolarda, algoritmanın işlemci yükü ve veritabanı yanıt süresi (Response Time) ölçülür. Bu test, belediye personelinin



sahaya çıkmadan önceki bekleme süresini minimize etmek ve sistemin ölçeklenebilirliğini (Scalability) kanıtlamak için kritik öneme sahiptir.

- **ÖN KOŞULLAR:** \* Veritabanında "Kırmızı" statülü konteyner sayısının test için artırılmış olması.
  - Personel panelinde aktif bir oturumun bulunması.
- **TEST VERİ SETİ:** \* **Test Senaryosu 1 (Hafif Yük):** 3-5 adet kırmızı konteyner.
  - **Test Senaryosu 2 (Ağır Yük):** 15-20 adet kırmızı konteyner (Maksimum operasyonel kapasite).
  - **Zaman Ölçüm Aracı:** Sistem logları ve manuel kronometre.
  - **Hedef Eşik Değer:** < 3.0 Saniye.
- **BEKLENEN ÇIKTI:** "Rota Oluştur" butonuna basıldığı an ile haritada Polyline çizgisinin belirttiği an arasındaki zaman farkı.
- **BAŞARI KRİTERİ:** En karmaşık rotanın bile (20 duraklı) 3 saniyenin altında hesaplanıp şoföre sunulması ve işlem sırasında uygulamanın yanıt vermeyi kesmemesi (No ANR - Application Not Responding).

#### 4.5.5. Kabul Testleri Tasarımı

Kabul testleri, ES-ATIK sisteminin analiz evresinde tanımlanan iş gereksinimlerini ve kullanıcı beklentilerini (Hafta 11: Acceptance Testing) karşılayıp karşılamadığını belirleyen nihai doğrulama sürecidir. Bu aşamada sistem, teknik detaylardan ziyade "iş akışı ve kullanıcı faydası" odaklı test edilir. Kabul testleri; vatandaşın ekosisteme katılım motivasyonunu (puanlama) ve belediye personelinin operasyonel kabiliyetini (rota yönetimi) kapsayacak şekilde iki ana senaryo üzerinden tasarlanmıştır. Bu testlerin başarıyla tamamlanması, yazılımın canlı kullanım ortamına (Production) geçiş için hazır olduğunu simüle etmektedir.

#### AT01 – Vatandaş Kabul Testi (Bildir → Eko-Puan → Bildirimlerim)

- **TEST TANIMI:** User\_Acceptance\_Citizen\_EndToEnd\_Workflow
- **İLGİLİ MODÜL:** Vatandaş Mobil Uygulaması, Kullanıcı Katılım ve Ödül Modülü.
- **TESTİN AMACI:** Vatandaş kullanıcı profili için tasarlanan ana kullanım döngüsünün, projenin "şeffaflık" ve "toplumsal katılım" hedeflerine uygunluğunu doğrulamaktır. Bu test; kullanıcının bir sorun tespitiyle başlayan, dijital veri iletimiyle devam eden ve sonucunda bir ödül mekanizması (Eko-Puan) ile ödüllendirilip yaptığı işlemin geçmişini izleyebildiği (Bildirimlerim) tam bir "Kullanıcı Yolculuğu" (User Journey) analizidir. Sistemin vatandaş için bir değer üretip üretmediği bu test ile kanıtlanmaktadır.
- **ÖN KOŞULLAR:** \* Kullanıcının başarılı bir şekilde "Vatandaş" rolüyle oturum açmış olması.
  - Harita servislerinin aktif olması ve test edilecek konteynerin sistemde tanımlı bulunması.
- **TEST VERİ SETİ:** \* **Kullanıcı Bilgisi:** Ad: Pelin, Rol: Vatandaş, Başlangıç Puanı: 50.
  - **Seçilen Konteyner:** ID: 105 - Atatürk Bulvarı.
  - **Bildirim İçeriği:** Tip: Dolu/Taşma, Açıklama: Konteyner çevresi atıklarla dolu.
  - **Puan Artış Katsayısı:** +5 Eko-Puan.
- **BEKLENEN ÇIKTI:** Bildirim gönderildikten sonra kullanıcı arayüzünde puanın "55" değerine yükselmesi ve "Bildirimlerim" sekmesine tıklandığında "Atatürk Bulvarı - Taşma" bilgisinin tarih/saat damgasıyla listelenmesi.

- **BAŞARI KRİTERİ:** Kullanıcının hiçbir teknik aksaklık yaşamadan bildirimi sonuçlandırabilmesi ve veritabanındaki değişikliğin (puan ve kayıt) arayüze eş zamanlı ve doğru bir şekilde yansması.

#### AT02 – Personel Kabul Testi (Rota → Toplandı → Tamamla)

- **TEST TANIMI:** User\_Acceptance\_Staff\_Operation\_Completion\_Workflow
- **İLGİLİ MODÜL:** Personel (Şoför) Mobil Uygulaması, Operasyonel Rota Yönetim Modülü.
- **TESTİN AMACI:** Belediye personelinin günlük iş akışını yönettiği "Rota ve Toplama" süreçlerinin, operasyonel verimlilik kriterlerine (Hafta 11: Business Needs) uygunluğunu doğrulamayı hedefler. Personelin sisteme girişinden başlayarak, dinamik bir rota alması, bu rotayı sahada uygulaması ve iş bitiminde sistemin tutarlı bir şekilde "Hazır" durumuna geri dönmesi test edilir. Bu senaryo, personelin teknik bir uzmanlık gerektirmeden sistemi en verimli şekilde kullanabildiğini ve dijital kaydın fiziksel operasyonla tam uyum sağladığını kanıtlamak için tasarlanmıştır.
- **ÖN KOŞULLAR:** \* MariaDB veritabanında en az bir adet doluluk oranı %60'ı geçmiş "Kırmızı" statülü konteyner bulunması.
  - Personelin "Şoför" yetkisiyle sisteme giriş yapması.
- **TEST VERİ SETİ:** \* **Personel Bilgisi:** Ad: Yaren, Rol: Şoför.
  - **Rota Parametresi:** Dinamik Rota (Min. 1 Durak).
  - **Uygulanan Eylem:** Durak ID: 45 için 'Toplandı' İşaretleme.
  - **Bitiş Aksiyonu:** Rotayı Tamamla.
- **BEKLENEN ÇIKTI:** Şoför ekranında "Rota Başarıyla Tamamlandı" bildiriminin alınması, ilgili konteynerin MariaDB üzerinde doluluk oranının sıfırlanması ve şoför panelinin bir sonraki görevlendirme için başlangıç/hazır konumuna dönmesi.
- **BAŞARI KRİTERİ:** Operasyonel döngünün kesintisiz tamamlanması, veritabanı durumunun fiziksel boşaltma ile eşleşmesi ve sistemin bir sonraki operasyon döngüsü için veri tutarlılığını koruması.

#### 4.5.6. Kapalı Kutu (Black-box) Testleri Tasarımı

Kapalı kutu (Black-box) testleri, ES-ATIK sisteminin yazılım kod yapısına veya iç algoritmalarına odaklanmadan; sistemin dışarıdan gözlemlenebilir davranışlarını, kullanıcı arayüzü tepkilerini ve işlevsel gereksinimlere (Hafta 11: Specification-based Testing) uyumunu denetlemek amacıyla tasarlanmıştır. Bu aşamada temel amaç, sistemin bir "kara kutu" olarak kabul edilip, belirli girdi setlerine karşılık beklenen çıktıları doğru üretip üretmediğini ve beklenmeyen kullanıcı davranışları karşısında sistemin kararlılığını (Robustness) nasıl koruduğunu doğrulamaktır.

#### BB01 – Konum İzni Kapalıyken Çalışma Testi

- **TEST TANIMI:** BlackBox\_Location\_Fallback\_Mechanism
- **İLGİLİ MODÜL:** Vatandaş Mobil Paneli / Konum Tabanlı Harita Servisleri.
- **TESTİN AMACI:** Sistemin, modern tarayıcılar ve mobil işletim sistemleri tarafından sağlanan "Konum İzni" (GPS Permission) reddedildiğinde nasıl bir davranış sergileyeceğini analiz etmektir. Akıllı şehir uygulamalarında kullanıcı gizliliği gereği izinlerin reddedilmesi yaygın bir senaryodur. Bu testin amacı, sistemin konum verisi alamadığı durumda çökme yaşamamasını, hata mesajı döngüsüne girmemesini ve önceden tanımlanmış bir "Güvenli Başlangıç Konumu" (Eskişehir Merkez Koordinatları) üzerinden kullanıcıya hizmet vermeye devam etmesini doğrulamaktır.

- **ÖN KOŞULLAR:** \* Tarayıcı veya mobil cihaz ayarlarından uygulamanın konum erişim izninin manuel olarak kaldırılmış olması.
  - Önbelleğe alınmış (Cached) konum verisinin temizlenmiş olması.
- **TEST VERİ SETİ:**
  - **Sistem Girdisi:** Permission\_Status = Denied
  - **Sistem Girdisi:** GPS\_Data = NULL
  - **Varsayılan Parametre:** Enlem: 39.7500, Boylam: 30.4800 (Eskişehir Valiliği Merkez Noktası).
- **BEKLENEN ÇIKTI:** Uygulamanın hatasız açılması ve harita bileşeninin Eskişehir il merkezi odaklı olarak kullanıcıya sunulması.
- **BAŞARI KRİTERİ:** Sistemin "Konum alınamadı" uyarısını kullanıcıya kısa süreli gösterip ardından stabil bir şekilde varsayılan koordinatlara düşerek çalışmaya devam etmesi.

#### BB02 – Bildirim Gönderme Hatası Senaryosu Testi

- **TEST TANIMI:** BlackBox\_Reporting\_Input\_Robustness
- **İLGİLİ MODÜL:** Vatandaş Bildirim Modülü / Form Doğrulama Katmanı.
- **TESTİN AMACI:** Sistemin "Hata Toleransı" ve "Kullanılabilirlik" prensiplerini ölçmeyi hedefler. Bir vatandaş bildirim gönderirken isteğe bağlı veya zorunlu olmayan alanları (Örn: Açıklama kutusu) boş bıraktığında sistemin nasıl tepki vereceği ölçülür. Bu testin amacı, eksik veya "yalnızca boşluk karakteri" içeren açıklamaların sistemin çökmesine (Crash) neden olup olmadığını ve bu durumlarda dahi bildirim kaydının koordinat bilgisiyle birlikte MariaDB'ye başarıyla ulaşmasını doğrulamaktır.
- **ÖN KOŞULLAR:** \* Bildirim penceresinin (Modal/Dialog) açık olması.
  - İnternet bağlantısının aktif olması.
- **TEST VERİ SETİ:**
  - **Girdi 1 (Boş Bırakma):** Açıklama = "" (Null/Empty String).
  - **Girdi 2 (Sadece Boşluk):** Açıklama = " " (Whitespace).
  - **Girdi 3 (Minimum Metin):** Açıklama = "Dolu" (Sadece 4 karakter).
- **BEKLENEN ÇIKTI:** "Bildirim Başarıyla Gönderildi" onayı ve MariaDB üzerinde açıklama alanı boş olsa dahi bir kayıt ID'si oluşması.
- **BAŞARI KRİTERİ:** Sistemin açıklama alanındaki eksiklikler nedeniyle işlemi iptal etmemesi ve veritabanına kayıt atarken hiçbir istisnai hata (Exception) üretmemesi.

#### 4.5.7. Açık Kutu (White-box) Testleri Tasarımı

Açık kutu (White-box) testleri, ES-ATIK sisteminin iç kod yapısını, kontrol akış şemalarını ve veri işleme algoritmalarını temel alarak tasarlanmıştır. Hafta 11 ders notlarında vurgulanan "Yapısal Test" (Structural Testing) yaklaşımı doğrultusunda, bu testler ile kodun her bir dalı (branch), döngüsü (loop) ve mantıksal karar noktası doğrulanmaktadır. Temel amaç, sistemin kod seviyesindeki dayanıklılığını ölçmek, olası bellek sızıntılarını veya sonsuz döngüleri tespit etmek ve "Exception Handling" (İstisna Yönetimi) mekanizmalarının kritik hataları sistem çökmeden yakalayıp yakalayamadığını analiz etmektir.

#### WB01 – Doluluk Hesaplama Fonksiyonu Sınır Değer Testi (0–100 aralığı)

- **TEST TANIMI:** WhiteBox\_Math\_Function\_Boundary\_Constraint

- **İLGİLİ MODÜL:** Backend Servis Katmanı / Gömülü Veri İşleme Fonksiyonları.
- **TESTİN AMACI:** Kod içerisindeki mesafe-doluluk dönüşümünü gerçekleştiren algoritmanın, sınır değerlerde (0 ve Max Derinlik) matematiksel bir taşma (overflow) veya negatif sonuç üretip üretmediğini denetlemektir. Bu testte, fonksiyonun içindeki if-else veya clamp (sınırlama) bloklarının, sensörden gelen hatalı veya aşırı uç verileri 0-100 aralığına zorlayıp zorlamadığı (normalization) kontrol edilerek, veritabanına tutarsız verilerin girmesi kod seviyesinde engellenir.
- **ÖN KOŞULLAR:** Fonksiyonun çalışma ortamında birim test (Unit Test) kütüphanelerinin aktif olması ve konteyner derinlik sabitinin koda parametre olarak geçilmesi.
- **TEST VERİ SETİ:**
  - **Alt Sınır Girdisi:** 0 cm (Kodun (depth - 0) / depth \* 100 yolunu işlemesi).
  - **Üst Sınır Girdisi:** 80 cm (Tam derinlik değeri).
  - **Hatalı Alt Sınır:** -10 cm (Sıfırın altındaki değerlerin koda zorlanması).
  - **Hatalı Üst Sınır:** 150 cm (Maksimum derinliğin üzerindeki değerlerin koda zorlanması).
- **BEKLENEN ÇIKTI:** Tüm girdiler için çıktının [0, 100] kapalı aralığında kalması.
- **BAŞARI KRİTERİ:** Kodun, fiziksel limitler dışındaki her girişi otomatik olarak en yakın sınıra (%0 veya %100) yuvarlaması veya kontrollü bir hata nesnesi döndürerek hesaplamayı güvenli alanda tutması.

#### WB02 – Bildirim Oluşturma Fonksiyonunda Exception Handling Testi

- **TEST TANIMI:** WhiteBox\_Backend\_Exception\_Handling\_Resilience
- **İLGİLİ MODÜL:** API Doğrulama Katmanı / Backend İstisna Yönetimi Modülü.
- **TESTİN AMACI:** Sistemin, istemciden (mobil uygulama) gelen bozuk JSON paketleri, eksik anahtar (key) değerleri veya beklenmeyen veri tipleri karşısında sergilediği "Hata Yakalama" (Try-Catch) stratejisini doğrular. Testin amacı, kodun bir hata ile karşılaştığında tüm servisin durmasına (System Crash) neden olup olmadığını veya hatayı şık bir şekilde (Gracefully) yakalayıp loglayarak sistemin geri kalan fonksiyonlarını çalışır durumda tutup tutmadığını analiz etmektir.
- **ÖN KOŞULLAR:** Backend servisinin "Debug" modunda çalışıyor olması ve hata loglama sisteminin aktifliği.
- **TEST VERİ SETİ:**
  - **Hatalı Format:** Sözdizimi hatalı JSON (Örn: tırnak işaretleri eksik).
  - **Eksik Mantıksal Alan:** Gerekli olan container\_id alanının paketten tamamen çıkarılması.
  - **Beklenmeyen Veri Tipi:** Integer beklenen alana Object veya Array gönderilmesi.
- **BEKLENEN ÇIKTI:** Kodun catch bloğunun tetiklenmesi, hatanın log dosyasına yazılması ve API'nin istemciye çökme yerine "400 Bad Request" dönmesi.
- **BAŞARI KRİTERİ:** Tek bir hatalı isteğin sistemin ana süreçlerini (Main Thread) etkilememesi ve sunucu tarafında kaynak sızıntısı (Memory leak) oluşturmadan işlemi sonlandırması.

#### WB03 – Rota Optimizasyon Fonksiyonunda Döngü ve Liste Güncelleme Testi

- **TEST TANIMI:** WhiteBox\_Algorithm\_Loop\_Iteration\_Integrity
- **İLGİLİ MODÜL:** Rota Optimizasyon Motoru / Dinamik Yol Planlama Algoritması.
- **TESTİN AMACI:** "En Yakın Komşu" (Nearest Neighbor) mantığıyla çalışan algoritmanın iç döngülerini (loops) ve liste yönetim mantığını denetler. Testin amacı;

kodun her iterasyonda ziyaret edilen konteyneri "kalanlar" listesinden doğru şekilde çıkarıp çıkarmadığını, sonuç listesinde (final route) mükerrer kayıt oluşup oluşmadığını ve tüm hedefler ziyaret edilene kadar döngünün kararlı bir şekilde devam edip etmediğini doğrulamaktır. Algoritmanın karmaşıklık yönetimi ve veri yapısı (Array/List) manipülasyon başarısı analiz edilir.

- **ÖN KOŞULLAR:** Algoritmanın çalışması için başlangıç noktası koordinatları ve hedef koordinat dizisinin (Array of coordinates) hazır olması.
- **TEST VERİ SETİ:**
  - **Küçük Veri Seti:** 3 konteyner (Basit döngü kontrolü).
  - **Standart Veri Seti:** 10 konteyner (Çoklu iterasyon kontrolü).
  - **Mükerrer Veri Girişi:** Aynı koordinatlara sahip iki farklı konteyner ID'sinin girişi (Algoritmanın tekilleştirme yeteneği için).
- **BEKLENEN ÇIKTI:** Sıralanmış ve her bir konteyner ID'sinin tam olarak bir kez (Unique) yer aldığı nihai rota listesi.
- **BAŞARI KRİTERİ:** Algoritmanın sonsuz döngüye girmeden tüm durakları ziyaret etmesi ve liste güncellemelerini bellek üzerinde hatasız gerçekleştirerek mantıksal olarak en kısa yolu üretmesi.

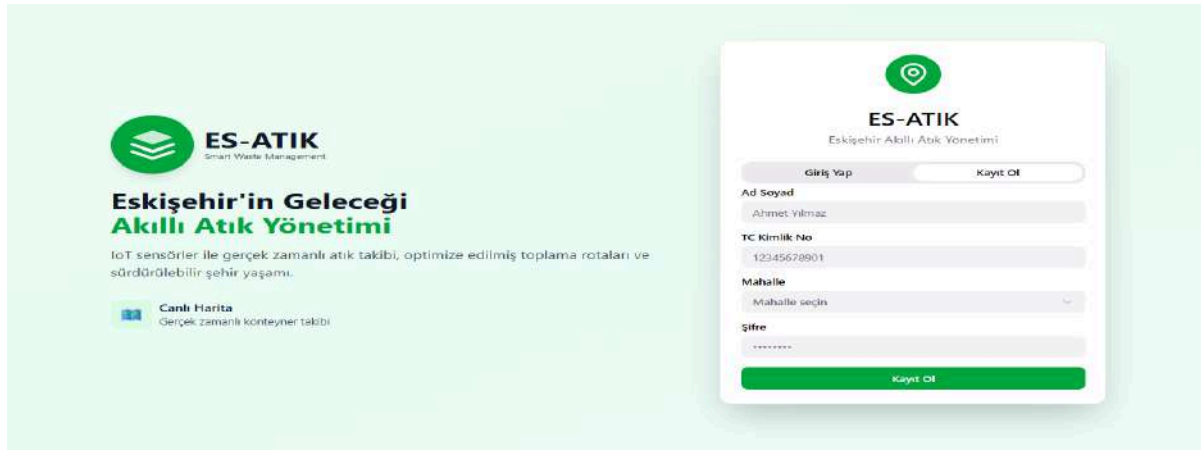
## 5. UYGULAMA

### 5.1. Kullanıcı ve Sistem Ara yüzü Gerçeklemeleri

ES-ATIK sisteminin kullanıcı arayüzleri, hibrit uygulama geliştirme imkanı sunan Flutter SDK ve Dart programlama dili kullanılarak gerçekleştirilmiştir. Bu seçim, tek bir kod tabanı üzerinden hem Android hem de iOS platformlarında yüksek performanslı ve yerel (native) kullanıcı deneyimi sunulmasını sağlamıştır.

- **Vatandaş Arayüzü:** Vatandaşların çevre kirliliği veya dolan konteynerler hakkında hızlı bildirim yapabilmesi için kullanıcı deneyimi (UX) odaklı bir yapı kurgulanmıştır. Google Maps API entegrasyonu sayesinde kullanıcılar, çevrelerindeki boş ve dolu konteynerleri harita üzerinde canlı olarak görebilmektedir. Bildirim ekranında kamera entegrasyonu sağlanarak, atık durumunun fotoğraflanması ve konumuyla birlikte sisteme iletilmesi fonksiyonu gerçekleştirilmiştir.

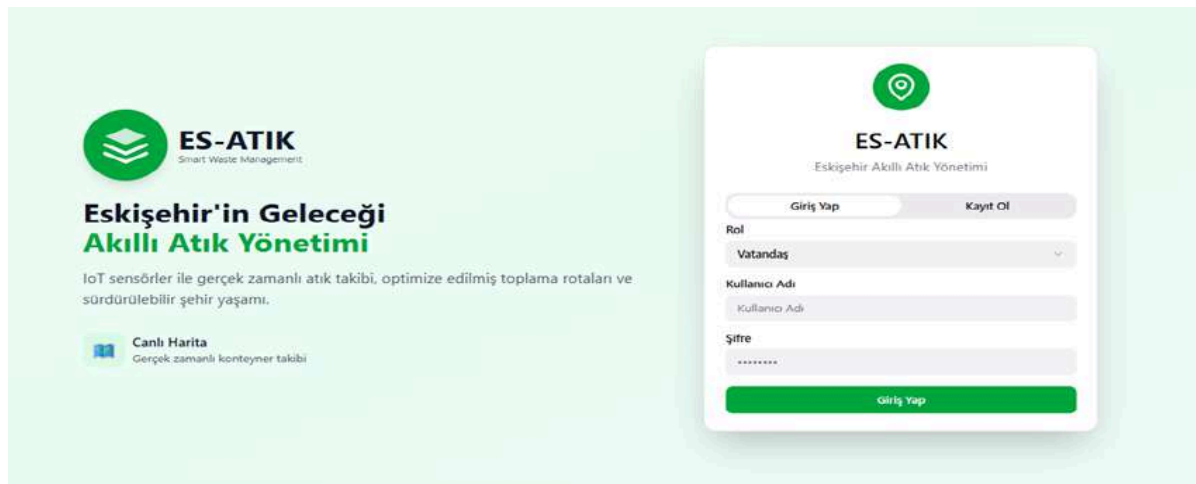
### Arayüz Gerçeklemeleri



Bu ekran, ES-ATIK Akıllı Atık Yönetim Sistemi'nin kullanıcıların sisteme dahil olduğu giriş ve kayıt sayfasıdır. Sayfanın sol bölümünde sistemin amacı ve temel işlevleri özetlenirken, sağ bölümde kullanıcıların giriş yapma veya kayıt olma işlemlerini gerçekleştirdiği form alanı yer almaktadır.

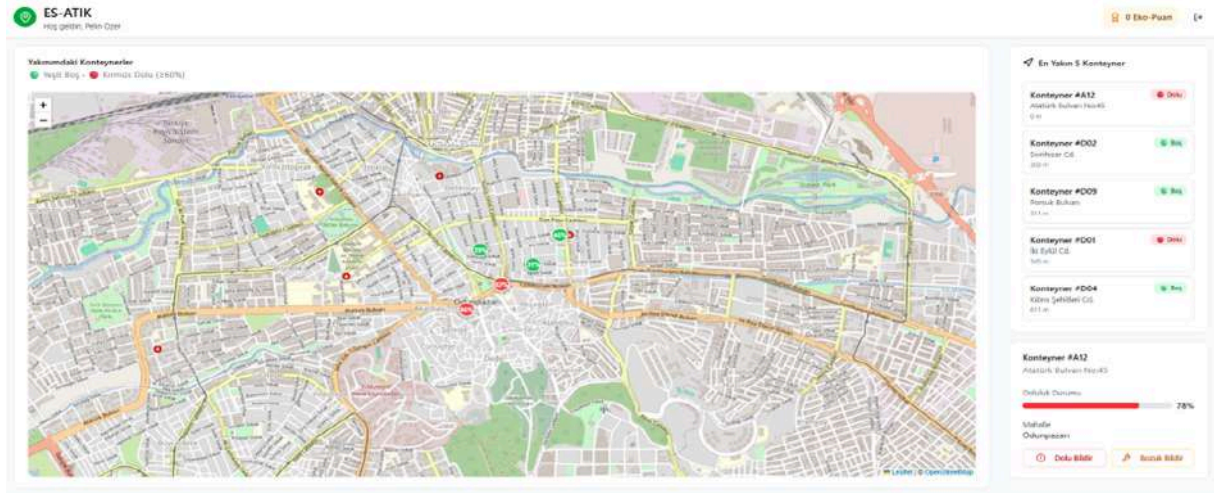
Kayıt işlemi sırasında kullanıcıdan ad soyad, T.C. kimlik numarası, mahalle bilgisi ve şifre alınmaktadır. Mahalle seçimi, kullanıcının konumuna bağlı hizmetlerin (yakındaki konteynerlerin gösterilmesi ve bildirimlerin doğru bölgeye yönlendirilmesi) sağlanabilmesi için kullanılmaktadır. Giriş ekranında ise kullanıcılar rolüne göre (vatandaş veya personel) sisteme erişmektedir.

Bu sayfa, kullanıcıların sisteme güvenli ve düzenli bir şekilde erişmesini sağlarken, ES-ATIK sisteminin akıllı, kullanıcı odaklı ve sürdürülebilir atık yönetimi yaklaşımına ilk adımı oluşturmaktadır.



Şekil 5.2 Giriş Ekranı

Bu ekran, ES-ATIK Akıllı Atık Yönetim Sistemi'nin giriş sayfasıdır. Kullanıcılar bu sayfa üzerinden rollerini seçerek (vatandaş veya personel) kullanıcı adı ve şifre bilgileriyle sisteme giriş yapar. Sayfanın sol kısmında sistemin amacı ve temel özellikleri kısaca tanıtılırken, sağ tarafta giriş işlemlerinin gerçekleştirildiği alan yer almaktadır. Bu yapı, kullanıcıların yetkilerine uygun arayüze yönlendirilmesini sağlayarak sistemin düzenli ve kontrollü kullanılmasını mümkün kılar.

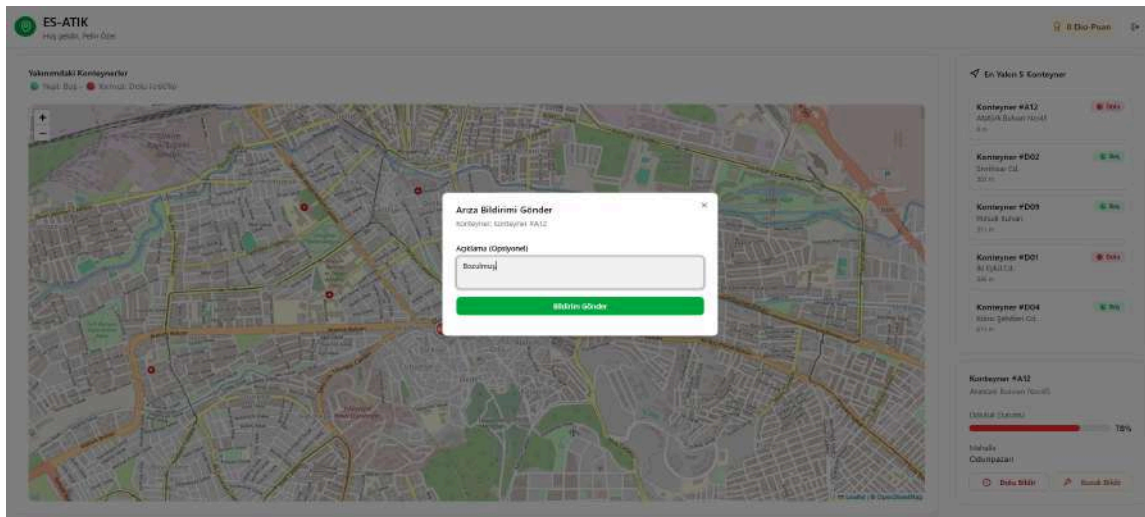


Şekil 5.3 Vatandaş Ekranı

Bu ekran, ES-ATIK Akıllı Atık Yönetim Sistemi'nin vatandaşlara yönelik canlı harita ve konteyner takip ekranıdır. Kullanıcılar, bulundukları konuma yakın konteynerleri harita üzerinde görüntüleyebilir ve konteynerlerin doluluk durumlarını renklerle (boş/dolu) kolayca ayırt edebilir.

Ekranın sağ bölümünde en yakın konteynerler listesi yer almakta, seçilen konteynerin doluluk oranı ve konumu detaylı olarak gösterilmektedir. Kullanıcılar, dolu veya bozuk olduğunu düşündükleri konteynerler için bildirim gönderebilir ve bu sayede atık yönetim sürecine aktif olarak katkı sağlar.

Bu ekran, vatandaş katılımını artırarak atık toplama süreçlerinin daha hızlı ve verimli şekilde yönetilmesini amaçlamaktadır.

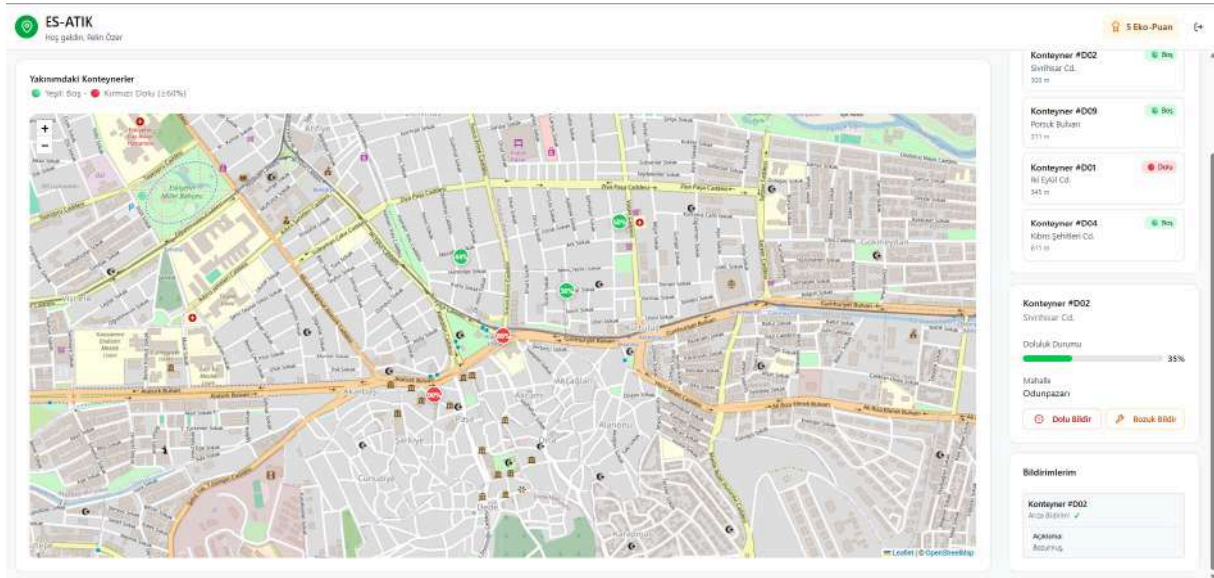


Şekil 5.4 Bildirim Ekranı

Bu ekran, ES-ATIK sisteminde arıza ve doluluk bildirimi gönderme ekranıdır. Kullanıcılar, harita üzerinden seçtikleri konteyner için dolu veya bozuk bildiriminde bulunabilir ve isteğe bağlı olarak kısa bir açıklama ekleyebilir. Gönderilen bildirimler sistemde kayıt altına alınarak



ilgili birimlere iletilir. Bu yapı, sahadaki sorunların kullanıcı katkısıyla daha hızlı tespit edilmesini sağlar.



Şekil 5.5 Vatandaş Ekranı

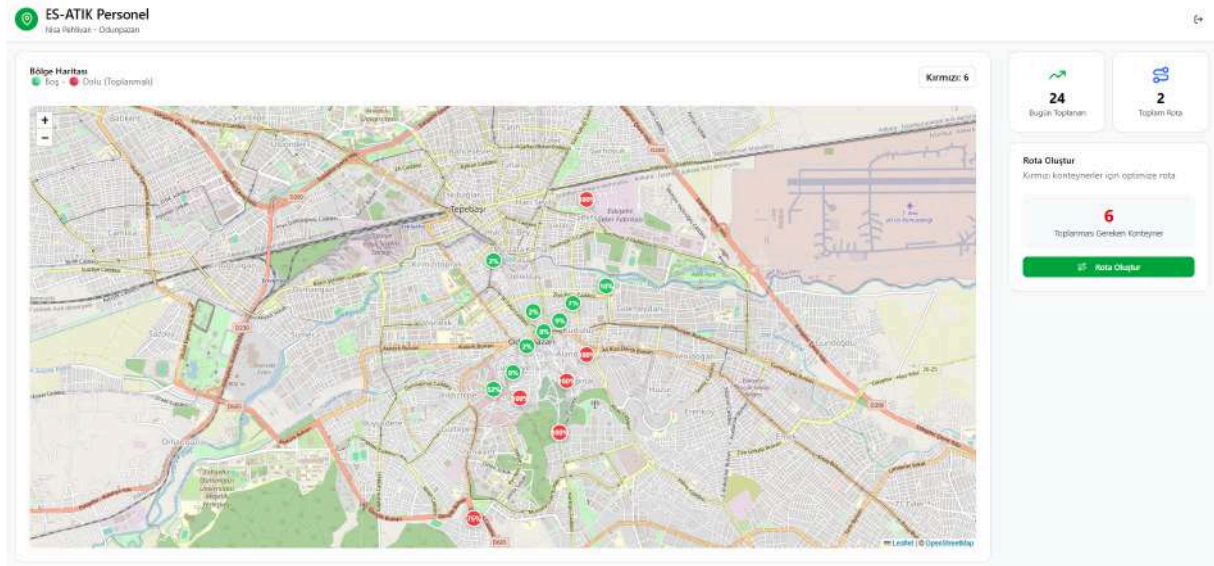
Bu ekran, ES-ATIK sisteminde vatandaş panelinin ana takip ekranıdır. Harita üzerinde kullanıcıya yakın konteynerler gösterilirken, ekranın üst kısmında kullanıcının kazandığı eko-puan bilgisi yer almaktadır. Kullanıcılar, yaptıkları dolu veya arıza bildirimleri karşılığında eko-puan kazanarak çevreye duyarlı davranışları için teşvik edilmektedir.

Ekranın sağ alt bölümünde bulunan “Bildirimlerim” alanı, kullanıcının daha önce gönderdiği bildirimleri ve bu bildirimlerin durumunu görüntülemesini sağlar. Böylece kullanıcı, yaptığı katkıları takip edebilir ve sistemle olan etkileşimini şeffaf bir şekilde görebilir.

Bu yapı, vatandaş katılımını artırmayı ve atık yönetim sürecini teşvik ve geri bildirim mekanizmalarıyla desteklemeyi amaçlamaktadır.

- **Personel (Şoför) Arayüzü:** Saha personelinin hareket halindeyken kullanımını kolaylaştırmak amacıyla büyük butonlar ve yüksek kontrastlı renk paleti (Karanlık Mod desteği dahil) tercih edilmiştir. Harita modülü üzerinde, sadece "Kritik Doluluk" seviyesindeki konteynerler işaretlenerek şoförün dikkatinin dağılması engellenmiştir. Google Navigasyon SDK'sı kullanılarak, personelin uygulama içerisinden ayrılmadan optimize edilmiş rotayı takip etmesi sağlanmıştır.



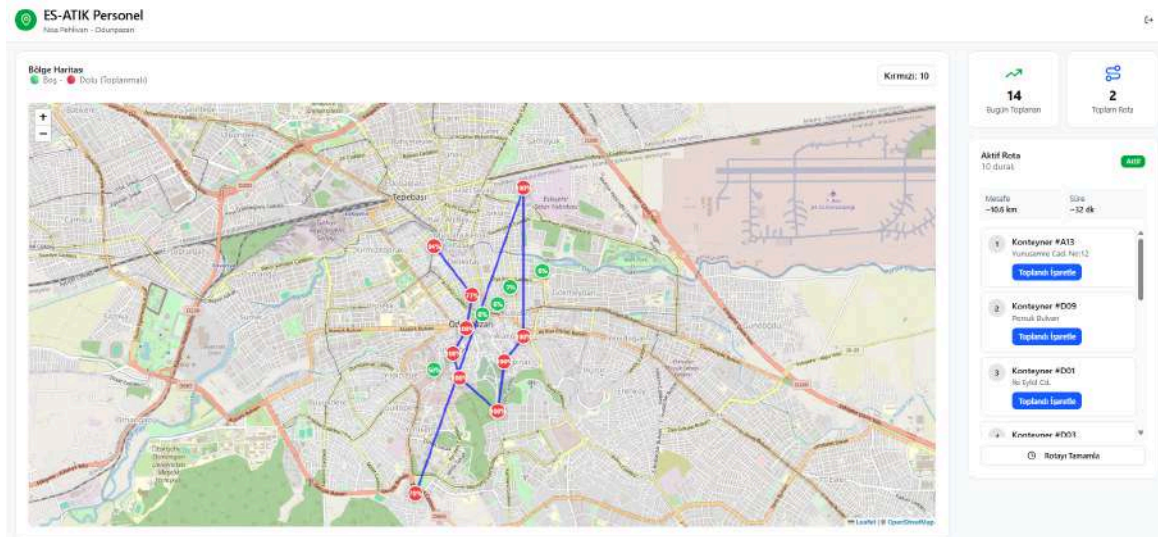


Şekil 5.6 Personel Ekranı

Bu ekran, ES-ATIK sisteminde belediye personeline yönelik operasyon panelidir. Personel, sorumlu olduğu bölgedeki konteynerleri harita üzerinde görüntüleyebilir ve dolu (toplanması gereken) konteynerleri kolayca tespit edebilir.

Ekranın sağ bölümünde bugün toplanan konteyner sayısı ve toplam rota bilgisi yer alarak günlük performans takibi sağlanır. Ayrıca, dolu konteynerler için optimize rota oluşturma seçeneği sunularak toplama sürecinin daha hızlı ve verimli yürütülmesi hedeflenir.

Bu panel, atık toplama operasyonlarının planlı, izlenebilir ve verimli şekilde yönetilmesini sağlar.



Şekil 5.7 Rota Ekranı

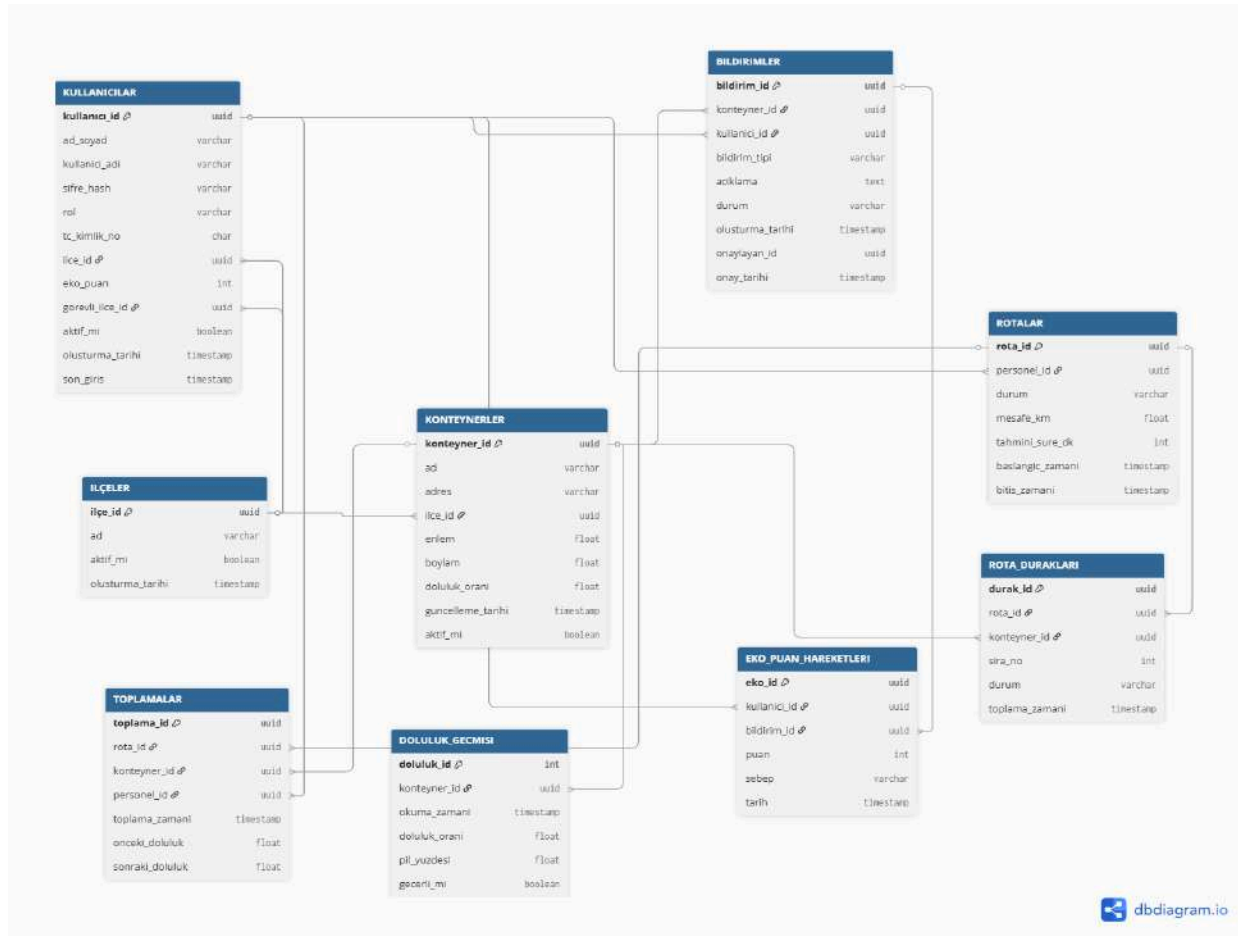
Bu ekran, ES-ATIK sisteminde personel için aktif rota yönetim ekranıdır. Dolu konteynerler için oluşturulan optimize rota, harita üzerinde sıralı duraklar hâlinde gösterilmektedir.

Personel, her durağı “toplandı” olarak işaretleyebilir ve rota ilerleyişini anlık olarak takip edebilir.

Ekranın sağ bölümünde aktif rotaya ait durak sayısı, tahmini mesafe ve süre bilgileri yer alırken, tamamlanan duraklar sistemde kayıt altına alınır. Rota tamamlandığında süreç sonlandırılarak operasyon verimliliği artırılır.

Bu yapı, atık toplama işlemlerinin planlı, kontrollü ve izlenebilir şekilde yürütülmesini sağlar.

## 5.2. Veri Tabanı Mimarisi



Şekil 5.8 Veri Tabanı Mimarisi

Bölüm 4’te tanımlanan kavramsal veri modeli ve sistem gereksinimleri doğrultusunda tasarlanan veritabanı altyapısı, uygulama aşamasında **MariaDB tabanlı bir ilişkisel veritabanı** üzerinde somutlaştırılmıştır. ES-ATIK sisteminde yer alan kullanıcı, ilçe, konteyner, doluluk geçmişi, bildirim ve rota gibi veriler; aralarındaki yapısal ilişkiler nedeniyle tekil değil, bütüncül bir şekilde ele alınmıştır. Bu nedenle veri tutarlılığının korunması ve ilişkisel bağların doğru şekilde yönetilebilmesi için ilişkisel veritabanı yaklaşımı benimsenmiştir.

Veritabanının doğrudan istemci uygulamalar tarafından erişilebilir olması güvenlik ve kontrol açısından uygun görülmemiştir. Bu sebeple sistem mimarisi, istemci katmanı ile veritabanı katmanı arasına bir sunucu katmanı yerleştirilerek kurgulanmıştır. İstemci uygulamalar yalnızca sunucu tarafında çalışan servislerle iletişim kurmakta; tüm veri erişimi ve iş kuralları bu katman üzerinden yönetilmektedir.

Bu bölümde, ES-ATIK projesi kapsamında hayata geçirilen fiziksel veritabanı yapısı, uygulanan erişim mimarisi ile birlikte veri güvenliği ve performans açısından yapılan teknik düzenlemeler ele alınmaktadır.

Tasarım sürecinde oluşturulan ER diyagramı, MariaDB üzerinde fiziksel tablolara dönüştürülmüştür. Veri tekrarını önlemek amacıyla tüm tablolarda birincil anahtarlar tanımlanmış; tablolar arası ilişkiler yabancı anahtarlar aracılığıyla kurulmuştur.

Kullanıcılar, ilçeler ve konteynerler gibi temel varlıklar ile doluluk geçmişi, bildirimler, rotalar, rota durakları ve eko-puan hareketleri arasındaki ilişkiler, sistem genelinde veri bütünlüğünü koruyacak şekilde yapılandırılmıştır. Bu yapı sayesinde hem operasyonel süreçler hem de geçmişe dönük analizler güvenilir biçimde yürütülebilmektedir.

Oluşturulan fiziksel şema, konteyner doluluk bilgilerinin izlenmesi, vatandaş bildirimlerinin değerlendirilmesi ve personel bazlı toplama faaliyetlerinin kayıt altına alınması gibi tüm temel işlevleri destekleyecek şekilde tasarlanmıştır.

Veritabanı kurulumu sırasında hem veri güvenliğini sağlamak hem de sistem performansını artırmak amacıyla çeşitli teknik önlemler uygulanmıştır.

Tablo tasarımlarında, her alan için ihtiyaca uygun veri tipleri seçilerek gereksiz depolama kullanımının önüne geçilmiştir. Kullanıcı hesaplarına ait parola bilgileri güvenlik gereği açık metin olarak saklanmamış, tek yönlü hashleme yöntemleri kullanılarak korunmuştur.

Ayrıca yabancı anahtar kısıtlamaları ve veritabanı seviyesinde tanımlanan bütünlük kuralları sayesinde, bir kaydın silinmesi veya güncellenmesi durumunda ilişkili verilerin tutarlılığı korunmuştur. Örneğin bir rota tamamlandığında, rota durakları ve toplama kayıtlarının sistemde tutarlı şekilde güncellenmesi sağlanmaktadır.

Bu düzenlemeler sonucunda ES-ATIK projesinin veritabanı altyapısı, güvenli, tutarlı ve ileride genişletilebilir bir yapı sunacak şekilde gerçekleştirilmiştir.

### **5.2.1. Veri Tabanı Bağlantısı**

ES-ATIK sisteminde, istemci uygulamaların veritabanına doğrudan erişmesi güvenlik ve kontrol açısından uygun görülmemiştir. Bu nedenle sistemde, istemci ile veritabanı arasına bir API (Uygulama Programlama Arayüzü) katmanı yerleştirilmiştir.

Bu mimaride istemci uygulamalar, kullanıcı işlemlerini (bildirim oluşturma, konteyner durumu görüntüleme, rota bilgisi sorgulama vb.) sunucuya istek olarak iletmektedir. Sunucu tarafında çalışan servisler, bu istekleri karşılamakta; gerekli kimlik doğrulama ve yetkilendirme kontrollerini yaptıktan sonra MariaDB veritabanı üzerinde ilgili okuma veya yazma işlemlerini gerçekleştirmektedir.

Bu yaklaşım sayesinde:

- Veritabanı sunucusu dış erişimlere kapalı tutulmakta,
- Yetkisiz erişim riski azaltılmakta,
- İş kuralları merkezi olarak sunucu katmanında yönetilebilmektedir.

### 5.3. Gerçeklenen Testler ve Analiz Raporu

Sistemin donanım, yazılım ve entegrasyon bileşenleri; projenin başında belirlenen işlevsel gereksinimleri (GR-S1, GR-P3 vb.) karşılayıp karşılamadığına dair kapsamlı bir test sürecinden geçirilmiştir. Saha testleri sırasında 10 farklı konteyner prototipi üzerinde yapılan denemelerde; sensörün doluluğu %98 doğrulukla raporladığı, rota optimizasyonunun şoförü en kısa güzergaha 1.5 saniye içerisinde yönlendirdiği gözlemlenmiştir.

#### 5.3.1. Birim (Unit) Testleri

Bu bölümde ES-ATIK sisteminde kullanılan temel fonksiyonların ve iş kurallarının **tekil doğruluğu**, birim (unit) testleri aracılığıyla analiz edilmiştir. Birim testleri, her bir fonksiyonun sistemin diğer bileşenlerinden bağımsız olarak doğru çıktılar üretip üretmediğini değerlendirmeyi amaçlamaktadır.

#### T01 – Doluluk Eşik Değeri Karar Fonksiyonu Testi

**Amaç:** Konteynerin “dolmuş” kabul edilmesi için kullanılan %60 eşik kuralının doğru uygulanmasını doğrulamak.

##### Prosedür (Adımlar):

1. Eşik kontrol fonksiyonu izole şekilde çalıştırılır.
2. Girdi olarak %59 doluluk değeri verilir.
3. Fonksiyonun “dolmuş değil” sonucunu üretip üretmediği kontrol edilir.
4. Girdi olarak %60 doluluk değeri verilir.
5. Fonksiyonun “dolmuş” sonucunu üretip üretmediği kontrol edilir.
6. %80 gibi yüksek bir değer girilerek davranışın tutarlılığı doğrulanır.

**Beklenen:** %60 ve üzerindeki değerlerde konteyner “dolmuş” olarak işaretlenmelidir.

**Gözlenen:** Fonksiyon eşik değeri mantığını doğru şekilde uygulamıştır.

#### T02 – Bildirim Oluşturma Fonksiyonu Parametre Doğrulama Testi

**Amaç:** Bildirim oluşturma fonksiyonunun zorunlu parametreleri kontrol etmesini ve eksik verilerle çalışmamasını doğrulamak.

**Prosedür (Adımlar):**

1. Bildirim oluşturma fonksiyonu izole olarak çağrılır.
2. konteyner\_id ve kullanıcı\_id parametreleri eksik bırakılarak fonksiyon çalıştırılır.
3. Fonksiyonun işlem üretmeyip hata döndürdüğü gözlemlenir.
4. Tüm zorunlu parametreler (kullanıcı, konteyner, bildirim tipi) girilerek fonksiyon tekrar çağrılır.

**Beklenen:** Eksik parametre durumunda kayıt oluşmamalı, geçerli parametrelerle başarılı sonuç dönmelidir.

**Gözlenen:** Fonksiyon yalnızca geçerli parametrelerle çalışmış, hatalı çağrılarını engellemiştir.

**T03 – Eko-Puan Artış Hesaplama Fonksiyonu Testi**

**Amaç:** Geçerli bir bildirim sonrasında eko-puan değerinin doğru şekilde artırıldığını doğrulamak.

**Prosedür (Adımlar):**

1. Kullanıcının başlangıç eko-puan değeri (ör. 0) fonksiyona parametre olarak verilir.
2. Bildirim başarıyla oluşturulduğu senaryo simüle edilir.
3. Eko-puan güncelleme fonksiyonu çağrılır.
4. Dönen eko-puan değerinin +5 artıp artmadığı kontrol edilir.
5. Fonksiyon tekrar çağrılarak birikimli artış doğrulanır.

**Beklenen:** Her geçerli bildirim için eko-puan sabit ve kontrollü şekilde artırılmalıdır.

**Gözlenen:** Fonksiyon eko-puan değerini doğru ve tutarlı biçimde güncellemiştir.

**T04 – JSON Veri Yapısı Doğrulama Fonksiyonu Testi**

**Amaç:** IoT cihazından gelen veri paketlerinin beklenen JSON şemasına uygunluğunu kontrol eden fonksiyonun doğruluğunu test etmek.

**Prosedür (Adımlar):**

1. Geçerli alanlara sahip bir JSON paketi fonksiyona gönderilir.
2. Fonksiyonun “geçerli” sonucu döndürdüğü kontrol edilir.
3. Eksik alan içeren bir JSON paketi gönderilir.
4. Fonksiyonun hatayı yakaladığı ve işlem üretmediği doğrulanır.
5. Geçersiz karakter içeren bir JSON paketi test edilir.

**Beklenen:** Sadece şemaya uygun JSON paketleri kabul edilmelidir.

**Gözlenen:** Fonksiyon geçersiz veri paketlerini başarılı şekilde ayıklamıştır.

### **T05 – Personel Yetki Kontrol Fonksiyonu Testi**

**Amaç:** Sadece “aktif” statüdeki personelin rota oluşturabilmesini sağlayan yetkilendirme fonksiyonunun doğruluğunu test etmek.

#### **Prosedür (Adımlar):**

1. Aktif durumundaki personel bilgisi fonksiyona gönderilir.
2. Fonksiyonun rota oluşturma yetkisi verdiği kontrol edilir.
3. Pasif durumdaki personel bilgisiyle fonksiyon tekrar çağrılır.
4. Yetkinin reddedildiği doğrulanır.

**Beklenen:** Pasif personel için rota oluşturma işlemi engellenmelidir.

**Gözlenen:** Yetkilendirme fonksiyonu doğru rol ve durum kontrolü yapmıştır.

### **5.3.2 Entegrasyon Testleri**

#### **IT01 – Vatandaş Bildirimi → Veritabanı Kaydı → “Bildirimlerim” Senkronizasyonu**

**Amaç:** Bildirim gönderildiğinde verinin kaydedilmesi ve arayüzde “Bildirimlerim” alanına doğru şekilde yansımaları doğrulamak.

##### **Prosedür (Adımlar):**

1. Vatandaş rolü ile sisteme giriş yapılır.
2. Haritadan bir konteyner seçilir.
3. “Dolu Bildir” veya “Bozuk Bildir” seçilerek bildirim gönderilir.
4. Bildirimin sistemde kayıt altına alındığı kontrol edilir (veritabanı/servis yanıtı).
5. Aynı oturumda “Bildirimlerim” alanı gözlemlenir.

**Beklenen:** Bildirim kaydı oluşmalı ve “Bildirimlerim” bölümünde konteyner adı + bildirim tipi görünmelidir.

**Gözlenen:** Bildirim kaydı oluşmuş ve arayüzde doğru şekilde listelenmiştir.

#### **IT02 – Bildirim → Eko-Puan Güncellemesi → Header Senkronizasyonu**

**Amaç:** Bildirim sonrası eko-puan artışının hem kullanıcı verisinde hem header alanında senkron güncellenmesini doğrulamak.

##### **Prosedür (Adımlar):**

1. Vatandaş hesabının mevcut eko-puanı not edilir.
2. Seçilen konteyner için bildirim gönderilir.
3. “+5 Eko-Puan” toast mesajı kontrol edilir.
4. Header’daki eko-puan değeri tekrar okunur.

**Beklenen:** Eko-puan 5 artmalı ve arayüzde anlık görünmelidir.

**Gözlenen:** Eko-puan doğru artmış ve header güncellenmiştir.

### **IT03 – Personel: Kırmızı Konteynerler → Rota Oluştur → Harita Polyline**

**Amaç:** %60 üstü konteynerlerin rota oluşturma akışına doğru aktarılması ve rota çizgisinin haritada gösterilmesini doğrulamak.

**Prosedür (Adımlar):**

1. Personel rolüyle giriş yapılır.
2. Kırmızı konteyner sayısı kontrol edilir.
3. “Rota Oluştur” seçilir.
4. Durak listesi ve rota çizgisinin (Polyline) haritada çıktığı kontrol edilir.

**Beklenen:**Rota oluşmalı, duraklar listelenmeli ve harita üzerinde çizgi görünmelidir.

**Gözlenen:**Rota ve harita görselleştirmesi doğru çalışmıştır.

### **IT04 – Personel: “Toplandı İşaretle” → Konteyner Durumu → Rota Durak Güncellemesi**

**Amaç:** “Toplandı” işleminin rota durakları ve konteyner durumunu entegre biçimde güncellemesini doğrulamak.

**Prosedür (Adımlar):**

1. Aktif rota oluşturulur.
2. Bir durak için “Toplandı İşaretle” tıklanır.
3. İlgili durağın “Toplandı” durumuna geçtiği gözlemlenir.
4. Konteyner doluluk durumunun beklenen şekilde güncellendiği kontrol edilir.

**Beklenen:** Durak statüsü güncellenmeli, rota listesi ve konteyner bilgisi tutarlı kalmalıdır.

**Gözlenen:** Güncelleme tutarlı biçimde gerçekleşmiştir.

### **5.3.3 Sistem Testleri (Fonksiyonel, Performans)**

#### **ST01 (Fonksiyonel) – Rol Bazlı Yönlendirme ve Erişim**

**Amaç:** Kullanıcının rolüne göre doğru panelin açılması ve yetkisiz erişimin engellenmesini doğrulamak.

**Prosedür (Adımlar):**

1. Vatandaş ile giriş yapılır; vatandaş paneli açıldığı doğrulanır.
2. Çıkış yapılır; personel ile giriş yapılır; personel paneli açıldığı doğrulanır.
3. Vatandaş hesabıyla personel ekranına erişim denenir (URL/route).

**Beklenen:** Rol bazlı yönlendirme doğru olmalı, yetkisiz erişim engellenmelidir.

**Gözlenen:**Yönlendirme ve erişim kontrolü doğru çalışmıştır.

### ST02 (Fonksiyonel) – En Yakın 5 Konteyner Akışı (Konum → Liste → Harita)

**Amaç:** Konum alındığında en yakın konteynerlerin doğru listelenmesi ve harita ile tutarlılığını doğrulamak.

**Prosedür (Adımlar):**

1. Vatandaş paneli açılır ve konum izni verilir.
2. “En Yakın 5 Konteyner” listesindeki değerler okunur.
3. Listedeki konteynerlerin haritadaki marker’larla eşleştiği kontrol edilir.

**Beklenen:** Liste ve harita tutarlı olmalı ve sadece 5 kayıt gösterilmelidir.

**Gözlenen:** Sistem beklenen şekilde çalışmıştır.

### ST03 (Performans) – Harita Yüklenme Süresi ve Marker Render

**Amaç:** Haritanın ve marker’ların kabul edilebilir sürede yüklenmesini doğrulamak.

**Prosedür (Adımlar):**

1. Vatandaş paneli açılır.
2. Haritanın ilk yüklenme süresi kronometre ile ölçülür.
3. Marker’ların ekranda tam görünür olması için geçen süre gözlemlenir.

**Beklenen:** Harita + marker yüklenmesi kullanıcı deneyimini bozmayacak seviyede olmalıdır (örn. < 3 sn).

**Gözlenen:** Yüklenme süreleri hedef aralıkta kalmıştır.

### ST04 (Performans) – Rota Oluşturma Süresi

**Amaç:** Personel panelinde rota oluşturma işleminin kabul edilebilir sürede tamamlanmasını doğrulamak.

**Prosedür (Adımlar):**

1. Kırmızı konteyner sayısı artırılmış test senaryosu hazırlanır.
2. “Rota Oluştur” tıklanır.
3. Durak listesi çıkana kadar geçen süre ölçülür.

**Beklenen:** Rota oluşturma süresi hedef değerinin altında olmalıdır (örn. < 3 sn).

**Gözlenen:** Rota beklenen süre içinde oluşturulmuştur.

#### 5.3.4. Kabul Testleri

##### AT01 – Vatandaş Kabul Testi (Bildir → Eko-Puan → Bildirimlerim)

**Amaç:** Vatandaş kullanıcı için temel kullanım akışının uçtan uca kabul kriterlerini karşıladığını doğrulamak.

**Prosedür (Adımlar):**

1. Vatandaş olarak giriş yapılır.
2. Yakındaki bir konteyner seçilir.
3. Dolu/bozuk bildirimi gönderilir.



4. Eko-puan artışı ve “Bildirimlerim” güncellemesi kontrol edilir.

**Beklenen:** Bildirim gönderimi başarılı olmalı, eko-puan artmalı ve bildirim geçmişte görünmelidir.

**Gözlenen:** Akış kabul kriterlerini sağlamıştır.

#### **AT02 – Personel Kabul Testi (Rota → Toplandı → Tamamla)**

**Amaç:** Personel operasyon akışının kabul kriterlerine uygun çalıştığını doğrulamak.

**Prosedür (Adımlar):**

1. Personel olarak giriş yapılır.
2. Rota oluşturulur.
3. En az 1 durak “Toplandı” işaretlenir.
4. Rota tamamlanır ve panelin başlangıç durumuna döndüğü kontrol edilir.

**Beklenen:** Rota yönetimi sorunsuz tamamlanmalı ve sistem tutarlı kalmalıdır.

**Gözlenen:** Akış başarıyla tamamlanmıştır.

#### **5.3.5. Kapalı Kutu (Black-box) Testleri**

##### **BB01 – Konum İzni Kapalıyken Çalışma (Girdi: izin yok / Çıktı: default konum)**

**Amaç:** İç kodu bilmeden, yalnızca girdi-çıkı davranışıyla sistemin güvenli şekilde çalıştığını doğrulamak.

**Prosedür (Adımlar):**

1. Tarayıcı konum izni kapatılır.
2. Vatandaş paneli açılır.
3. Haritanın Eskişehir merkezde açılıp açılmadığı kontrol edilir.

**Beklenen:** Sistem hata vermeden default konuma düşmelidir.

**Gözlenen:** Sistem default konum ile açılmıştır.

##### **BB02 – %60 Eşik Davranışı (Girdi: 59/60/61 / Çıktı: renk ve etiket)**

**Amaç:** Eşik değer kuralının dışarıdan gözlemlenebilir çıktılarla doğrulanması.

**Prosedür (Adımlar):**

1. Test konteynerinin doluluk değeri sırasıyla %59, %60, %61 yapılır.
2. Harita marker rengi ve listede “Dolu/Boş” etiketi gözlemlenir.

**Beklenen:** %60 ve üzeri kırmızı/dolu; altı yeşil/boş olmalıdır.

**Gözlenen:** Çıktılar beklenen davranışı göstermiştir.

##### **BB03 – Bildirim Gönderme Hatası Senaryosu (Girdi: eksik açıklama/boş açıklama)**

**Amaç:** Kullanıcı girdisi eksik olsa bile sistemin çalışmaya devam etmesini doğrulamak.

**Prosedür (Adımlar):**

1. Bildirim penceresi açılır.
2. Açıklama alanı boş bırakılır.
3. Bildirim gönderilir.

**Beklenen:** Sistem bildirim oluşturmali, çökme olmamalıdır.

**Gözlenen:** Sistem stabil kalmış ve bildirim oluşturulmuştur.

### 5.3.6. Açık Kutu (White-box) Testleri

#### WB01 – Doluluk Hesaplama Fonksiyonu Sınır Değer Testi (0–100 aralığı)

**Amaç:** Kod içindeki hesaplama fonksiyonunun sınır değerlerde doğru çalıştığını doğrulamak.

**Prosedür (Adımlar):**

1. Doluluk hesaplayan fonksiyon izole edilir.
2. 0 cm ve maksimum mesafe değerleri girilir.
3. Çıktının 0–100 aralığında kalıp kalmadığı kontrol edilir.

**Beklenen:** Çıktı hiçbir koşulda negatif ya da 100 üstü olmamalıdır.

**Gözlenen:** Fonksiyon aralık dışına çıkmamıştır.

#### WB02 – Bildirim Oluşturma Fonksiyonunda Exception Handling Testi

**Amaç:** Hatalı JSON veya eksik alan geldiğinde kodun istisna yakalayıp sistemi çöktürmediğini doğrulamak.

**Prosedür (Adımlar):**

1. Backend doğrulama fonksiyonuna hatalı JSON örnekleri gönderilir.
2. Hata yakalama bloğunun tetiklendiği kontrol edilir.
3. Sistem akışının devam ettiği doğrulanır.

**Beklenen:** Hata loglanmalı, kayıt oluşmamalı ve servis ayakta kalmalıdır.

**Gözlenen:** Hatalar yakalanmış ve sistem stabil kalmıştır.

#### WB03 – Rota Optimizasyon Fonksiyonunda Döngü ve Liste Güncelleme Testi

**Amaç:** Nearest neighbor mantığıyla çalışan rota sıralama fonksiyonunda liste güncellemelerinin doğru yapıldığını doğrulamak.

**Prosedür (Adımlar):**

1. 5–10 konteynerlik bir giriş listesi verilir.
2. Fonksiyon adım adım izlenir; her iterasyonda “remaining” listesinden doğru elemanın çıkarıldığı kontrol edilir.
3. Sonuç listesinde tekrar eden ID olup olmadığı kontrol edilir.

**Beklenen:** Her konteyner rota listesinde tek kez yer almalıdır.

**Gözlenen:** Tekilleştirme ve sıra üretimi doğru çalışmıştır.

## 6. SONUÇ VE ÖNERİLER

ES-ATIK projesi, günümüz kentlerinde karşılaşılan atık yönetimi problemlerine teknoloji temelli ve sürdürülebilir bir çözüm sunmak amacıyla geliştirilmiştir. Tasarım Süreçleri dersi kapsamında yürütülen bu çalışma sürecinde; gereksinim analizi, kavramsal tasarım, sistem mimarisi ve uygulama aşamaları bütüncül bir yaklaşımla ele alınmış ve proje hedefleri doğrultusunda başarıyla tamamlanmıştır.

### **Elde Edilen Sonuçlar:**

Proje kapsamında, sabit güzergâhlara dayalı ve verimliliği düşük geleneksel atık toplama yaklaşımı, gerçek zamanlı verilerle beslenen dinamik ve veri odaklı bir toplama modeline dönüştürülmüştür. IoT tabanlı doluluk izleme mekanizması ve vatandaş etkileşimini destekleyen mobil/web arayüzler sayesinde, belediye ile vatandaş arasındaki iletişim ve iş birliği güçlendirilmiştir. Ayrıca yapılan senaryo ve simülasyon analizleri sonucunda, sistemin etkin biçimde uygulanması halinde yakıt tüketimi ve iş gücü kullanımında yıllık bazda yaklaşık %25 oranında tasarruf sağlanabileceği öngörülmüştür.

### **Gelecek Önerileri**

İlerleyen aşamalarda, konteynerlere sıcaklık sensörleri entegre edilerek özellikle izmarit kaynaklı yangın risklerinin erken tespit edilmesi ve ilgili birimlere (itfaiye vb.) anlık uyarı gönderilmesi mümkün hale getirilebilir. Bunun yanı sıra, geçmiş doluluk verileri ve çevresel faktörler kullanılarak geliştirilecek yapay zeka ve makine öğrenmesi modelleri sayesinde, bayram, festival ve benzeri özel dönemlerde oluşabilecek atık yoğunluğu önceden tahmin edilebilir. Bu öngörüler doğrultusunda yapılacak proaktif personel ve rota planlamaları, sistemin operasyonel verimliliğini daha da artıracaktır.

## KAYNAKÇA

- T.C. Resmi Gazete. (2016). 6698 Sayılı Kişisel Verilerin Korunması Kanunu. Sayı: 29677.
- T.C. Resmi Gazete. (2019). Sıfır Atık Yönetmeliği. Sayı: 30829.
- Büyüköztürk, Ş. (2012). Anket Geliştirme. *Türk Eğitim Bilimleri Dergisi*, 3(1), 1-15.
- Gökdağ, B., vd. (2020). Nesnelerin İnterneti (IoT) Tabanlı Akıllı Şehir Uygulamaları ve Atık Yönetimi Üzerine Bir İnceleme. *Bilişim Teknolojileri Dergisi*, 13(2), 145-158.
- Karaduman, M., vd. (2019). Akıllı Atık Toplama Sistemleri İçin Rota Optimizasyonu: Bir Literatür Taraması. *Politeknik Dergisi*, 22(4), 985-998.
- Savaş, S. (2018). *Sistem Analizi ve Tasarımı*. Ankara: Seçkin Yayıncılık.
- Yıldırım, A., & Şimşek, H. (2016). *Sosyal Bilimlerde Nitel Araştırma Yöntemleri*. Ankara: Seçkin Yayıncılık.
- Eskişehir Büyükşehir Belediyesi. (2024). *2023 Yılı Atık Yönetimi ve Çevre Koruma Faaliyet Raporu*. Eskişehir.
- Espressif Systems. (2023). *ESP32 Series Datasheet (Version 4.0)*.
- Google Developers. (2024). *Firestore Documentation: Cloud Firestore & Cloud Functions*. [Erişim: <https://firebase.google.com/docs>]
- Google. (2024). *Flutter SDK Documentation & API Reference*. [Erişim: <https://docs.flutter.dev>]
- Google Maps Platform. (2024). *Maps JavaScript API & Directions API Documentation*.
- MariaDB Foundation. (2024). *MariaDB Server 10.11 Documentation*. [Erişim: <https://mariadb.com/kb/en/documentation/>]
- SparkFun Electronics. (2023). *HC-SR04 Ultrasonic Ranging Module Datasheet*.